

TỔNG QUAN VỀ NGÔN NGỮ TRUY VẤN CÓ CẤU TRÚC

SQL – STRUCTURED QUERY LANGUAGE

MỤC LỤC

I. Phân loại lệnh SQL	2
II. Ngôn ngữ định nghĩa dữ liệu (DDL)	3
1. CREATE TABLE - Tạo bảng mới	3
2. DROP TABLE - Xóa bảng	4
3. SQL TRUNCATE TABLE	4
4. ALTER TABLE - Sửa đổi cấu trúc bảng	4
III. Ngôn ngữ Thao tác Dữ liệu (DML)	6
1. INSERT INTO - Thêm dữ liệu	6
2. UPDATE - Cập nhật dữ liệu	8
3. DELETE - Xóa dữ liệu	9
IV. Lệnh Truy vấn Dữ liệu (SELECT)	10
1. FROM và JOIN	10
1.1. INNER JOIN (Phép nối trong)	10
1.2. LEFT JOIN (Phép nối ngoài bên trái)	12
1.3. RIGHT JOIN (Phép nối ngoài bên phải)	13
1.4. FULL OUTER JOIN (Phép nối ngoài đầy đủ)	14
1.5. CROSS JOIN (Tích Đề-các)	15
1.6. Các phép NATURAL JOIN	16
Student Table	16
2. WHERE	17
2.1. ALL	17
2.2. AND	18
2.3. OR	19
2.4. NOT	19
2.5. IN	19
2.6. BETWEEN	20
2.7. LIKE	20
2.8. IS NULL/ IS NOT NULL	22
2.9. ANY	22
2.10. EXISTS	23
3. GROUP BY	24
4. HAVING	27
5. SELECT	28
6. ORDER BY	29
7. Subqueries (Truy vấn con)	29
8. Các phép toán khác	30
8.1. Union $\cup$	30
8.2. Intersection $\cap$	33
8.3. Difference $-$	35
9. Conditions Involving Tuples (Điều kiện liên quan đến các bộ dữ liệu)	36
10. Correlated Subqueries (truy vấn con tương quan)	37

V. Ràng buộc Toàn vẹn (Integrity Constraints)	38
1. NOT NULL	40
2. UNIQUE	40
3. PRIMARY KEY (Khóa chính)	41
4. FOREIGN KEY (Khóa ngoại)	43
5. CHECK	45
6. DEFAULT	47
VI. Views	49
VII. Renaming	51
VIII. Indexes (Chỉ mục)	52
IX. Auto Increment	54

- **Cơ sở dữ liệu** giống như một tủ hồ sơ được tổ chức cực kỳ khoa học, bao gồm các thành phần:
 - **Bảng (Table):** Mỗi ngăn tủ là một bảng. Ví dụ: ngăn SinhVien, ngăn LopHoc.
 - **Hàng (Row) hay Bản ghi (Record):** Mỗi bộ hồ sơ trong ngăn là một hàng. Ví dụ: hồ sơ của sinh viên "Nguyễn Văn A".
 - **Cột (Column) hay Trường (Field):** Mỗi thông tin trên một bộ hồ sơ là một cột. Ví dụ: MaSinhVien, HoTen, NgaySinh.
- **SQL (Structured Query Language)** chính là ngôn ngữ bạn dùng để ra lệnh cho người quản lý tủ hồ sơ (Hệ Quản trị CSDL - DBMS) thực hiện các công việc như: tạo ngăn tủ mới (tạo bảng), thêm hồ sơ, tìm kiếm thông tin/truy vấn, cập nhật, hoặc xóa dữ liệu, v.v.

I. Phân loại lệnh SQL

SQL có thể được chia thành các nhóm ngôn ngữ con chính:

- DDL (Data Definition Language - Ngôn ngữ Định nghĩa Dữ liệu):** Dùng để định nghĩa và quản lý **cấu trúc của cơ sở dữ liệu**, bao gồm các lệnh:
 - **CREATE:** Tạo mới cơ sở dữ liệu, bảng, view, index...
 - **ALTER:** Thay đổi cấu trúc của các đối tượng đã có.
 - **DROP:** Xóa các đối tượng khỏi cơ sở dữ liệu.
- DML (Data Manipulation Language - Ngôn ngữ Thao tác Dữ liệu):** Dùng để quản lý **dữ liệu bên trong các bảng**, bao gồm các lệnh:
 - **INSERT:** Chèn dữ liệu mới vào bảng.
 - **UPDATE:** Cập nhật dữ liệu đã có.
 - **DELETE:** Xóa dữ liệu khỏi bảng.
 - **SELECT:** Truy vấn và lấy dữ liệu từ cơ sở dữ liệu (đôi khi được xếp vào một nhóm riêng là DQL - Data Query Language).
- DCL (Data Control Language - Ngôn ngữ Điều khiển Dữ liệu):** Dùng để **quản lý quyền truy cập**.
 - **GRANT:** Cấp quyền cho người dùng.
 - **REVOKE:** Thu hồi quyền đã cấp.

4. TCL (Transaction Control Language - Ngôn ngữ Điều khiển Giao tác): Dùng để quản lý các giao tác trong cơ sở dữ liệu.

- **COMMIT:** Lưu lại các thay đổi của một giao dịch.
- **ROLLBACK:** Hủy bỏ các thay đổi trong một giao dịch.
- **SAVEPOINT:** Đặt một điểm lưu tạm thời trong một giao dịch.

II. Ngôn ngữ định nghĩa dữ liệu (DDL)

1. CREATE TABLE - Tạo bảng mới

- Lệnh này dùng để tạo một bảng mới trong cơ sở dữ liệu. Bạn phải định nghĩa rõ các cột thông tin sẽ có trong đó.

- **Cú pháp:**

```
CREATE TABLE ten_bang (  
    ten_cot1 kieu_du_lieu [rang_buoc_cot],  
    ten_cot2 kieu_du_lieu [rang_buoc_cot],  
    ...  
    [rang_buoc_bang]  
);
```

- *ten_bang*: Tên của bảng bạn muốn tạo.
- *ten_cot*: Tên của các cột trong bảng.
- *kieu_du_lieu*: Loại dữ liệu mà cột có thể chứa (ví dụ: INT, TEXT, VARCHAR(255), REAL, DATE, BOOLEAN).
- *rang_buoc_cot/bang*: Các quy tắc áp dụng cho dữ liệu (**sẽ được giải thích trong mục Ràng buộc Toàn vẹn**).

- **Ví dụ:** Tạo bảng Company (Công ty).

```
CREATE TABLE Company (  
    ID INT PRIMARY KEY,          - Mã công ty, là khóa chính  
    Name TEXT NOT NULL,          - Tên công ty, không được để trống  
    Age INT NOT NULL,            - Tuổi  
    Address CHAR(50),            - Địa chỉ, tối đa 50 ký tự  
    Salary REAL                  - Lương, kiểu số thực  
);
```

- **NOT NULL:** Đảm bảo cột Name và Age phải có giá trị, nghĩa là không thể chèn bản ghi mới/cập nhật bản ghi mà không thể không thêm giá trị cho các trường này.

- **Create Table Using Another Table**

- A copy of an existing table can also be created using **CREATE TABLE**.
- The **new table gets the same column definitions**. All columns or specific columns can be selected.
- If you create a new table using an existing table, the new table will be filled with the existing values from the old table.

- **Syntax**

```
CREATE TABLE new_table_name AS
SELECT column1, column2,...
FROM existing_table_name
WHERE ....;
```

- The following SQL creates a new table called "TestTable" (which is a copy of the "Customers" table):

```
CREATE TABLE TestTable AS
SELECT customername, contactname
FROM customers;
```

2. DROP TABLE - Xóa bảng

- Lệnh này dùng để **xóa hoàn toàn một bảng và tất cả dữ liệu** bên trong nó. **Lệnh này cực kỳ nguy hiểm, không thể hoàn tác!**

```
DROP TABLE ten_bang;
```

3. SQL TRUNCATE TABLE

- The **TRUNCATE TABLE** statement is used to **delete the data** inside a table, but **not the table** itself.

```
TRUNCATE TABLE table_name;
```

- Nếu có trường ID được tạo tự động, nó tự reset về 1.

4. ALTER TABLE - Sửa đổi cấu trúc bảng

- Lệnh này dùng để sửa đổi cấu trúc của một bảng đã tồn tại, như thêm, xóa, hoặc sửa đổi cột.
- The ALTER TABLE statement is also used to **add and drop various constraints** on an existing table.

- **Cú pháp:**

- **Thêm một cột mới**

```
ALTER TABLE ten_bang
ADD ten_cot_moi kieu_du_lieu;
```

- **Xóa một cột**

```
ALTER TABLE ten_bang
DROP COLUMN ten_cot;
```

- **Sửa đổi kiểu dữ liệu của một cột (cú pháp có thể khác nhau giữa các hệ CSDL)**

- **SQL Server / MS Access:**

```
ALTER TABLE table_name
ALTER COLUMN column_name datatype;
```

- **My SQL / Oracle (prior version 10G):**

```
ALTER TABLE table_name
MODIFY COLUMN column_name datatype;
```

- **Oracle 10G and later:**

```
ALTER TABLE table_name
MODIFY column_name datatype;
```

- **ALTER TABLE - RENAME COLUMN:**

```
ALTER TABLE table_name
RENAME COLUMN old_name to new_name;
```

- **Example:** Look at the "Persons" table:

ID	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

- Now we want to **add a column** named "DateOfBirth" in the "Persons" table. We use the following SQL statement:

```
ALTER TABLE Persons
ADD DateOfBirth date;
```

- Notice that the new column, "DateOfBirth", is of type date and is going to hold a date. The data type specifies what type of data the column can hold.
- The "Persons" table will now look like this:

ID	LastName	FirstName	Address	City	DateOfBirth
1	Hansen	Ola	Timoteivn 10	Sandnes	
2	Svendson	Tove	Borgvn 23	Sandnes	
3	Pettersen	Kari	Storgt 20	Stavanger	

- Now we want to **change the data type of the column named "DateOfBirth"** in the "Persons" table. We use the following SQL statement:

```
ALTER TABLE Persons
ALTER COLUMN DateOfBirth year;
```

- Notice that the "DateOfBirth" column is now of type year and is going to hold a year in a **two- or four-digit format**.
- Next, we want to **delete the column named "DateOfBirth"** in the "Persons" table. We use the following SQL statement:

```
ALTER TABLE Persons
DROP COLUMN DateOfBirth;
```

- The "Persons" table will now look like this:

ID	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

III. Ngôn ngữ Thao tác Dữ liệu (DML)

1. INSERT INTO - Thêm dữ liệu

- Dùng để chèn một hoặc nhiều hàng mới vào một bảng.
- **Chèn một hàng (Specify both the column names and the values to be inserted)**

```
INSERT INTO ten_bang (cot1, cot2, ...)  
VALUES (gia_tri1, gia_tri2, ...);
```

- **Chèn nhiều hàng**

```
INSERT INTO ten_bang (cot1, cot2, ...)  
VALUES  
    (gia_tri_hang1_1, gia_tri_hang1_2, ...),  
    (gia_tri_hang2_1, gia_tri_hang2_2, ...),  
    ...;
```

- If you are **adding values for all the columns** of the table, you **do not need to specify the column names** in the SQL query. However, make sure the order of the values is in the same order as the columns in the table.

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

- **Example:** Below is a selection from the **Customers** table used in the examples:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland

- **Inserts a new record** in the "Customers" table:

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country)  
VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');
```

- The selection from the "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland
92	Cardinal	Tom B. Erichsen	Skagen 21	Stavanger	4006	Norway

- **Did you notice that we did not insert any number into the CustomerID field?** The CustomerID column is an [auto-increment](#) field and will be generated automatically when a new record is inserted into the table. **(Xem phần AUTO INCREMENT)**
- It is also possible to only **insert data in specific columns**. The following SQL statement will insert a new record, but only insert data in the "CustomerName", "City", and "Country" columns (CustomerID will be updated automatically):

```
INSERT INTO Customers (CustomerName, City, Country)
VALUES ('Cardinal', 'Stavanger', 'Norway');
```

- The selection from the "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland
92	Cardinal	null	null	Stavanger	null	Norway

- It is also possible to insert multiple rows in one statement. To insert multiple rows of data, we use the same **INSERT INTO** statement, **but with multiple values**:

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country)
VALUES
('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway'),
('Greasy Burger', 'Per Olsen', 'Gateveien 15', 'Sandnes', '4306', 'Norway'),
('Tasty Tee', 'Finn Egan', 'Streetroad 19B', 'Liverpool', 'L1 0AA', 'UK');
```

- Make sure you **separate each set of values with a comma** . The selection from the "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland
92	Cardinal	Tom B. Erichsen	Skagen 21	Stavanger	4006	Norway
93	Greasy Burger	Per Olsen	Gateveien 15	Sandnes	4306	Norway
94	Tasty Tee	Finn Egan	Streetroad 19B	Liverpool	L1 0AA	UK

2. UPDATE - Cập nhật dữ liệu

- Dùng để cập nhật các hàng dữ liệu **đã tồn tại** trong bảng.
- Cú pháp:**

```
UPDATE ten_bang
SET    cot1 = gia_tri_moi1,
       cot2 = gia_tri_moi2
WHERE dieu_kien;
```

- Quan trọng:** Mệnh đề WHERE xác định những hàng nào sẽ được cập nhật. Nếu bỏ qua WHERE, **tất cả** các hàng trong bảng sẽ bị cập nhật.
- Ví dụ:** Below is a selection from the **Customers** table used in the examples:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

- The following SQL statement **updates the first customer (CustomerID = 1) with a new contact person and a new city.**

```
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;
```

- The selection from the "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

- The following SQL statement will **update the ContactName to "Juan" for all records where country is "Mexico":**


```
UPDATE Customers
SET ContactName='Juan'
WHERE Country='Mexico';
```

- o The selection from the "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Juan	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Juan	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

- o If you omit the **WHERE** clause, ALL records will be updated!

```
UPDATE Customers
SET ContactName='Juan';
```

- o The selection from the "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Juan	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Juan	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Juan	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Juan	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Juan	Berguvsvägen 8	Luleå	S-958 22	Swed

3. DELETE - Xóa dữ liệu

- Dùng để xóa các hàng dữ liệu **đã tồn tại** khỏi bảng.
- **Cú pháp:**

```
DELETE FROM ten_bang WHERE dieu_kien;
```

- **Quan trọng:** Tương tự UPDATE, nếu bỏ qua mệnh đề WHERE, **tất cả các hàng trong bảng sẽ bị xóa.**
- Nếu ID được tạo tự động, giá trị của trường ID cho bản ghi mới sẽ không reset về 1, mà tăng lên 1 so với ID của bản ghi vừa xóa.
- **Ví dụ:** Xóa các nhân viên có tên chứa "Santana".

```
DELETE FROM employees
WHERE name LIKE '%Santana%';
```

IV. Lệnh Truy vấn Dữ liệu (SELECT)

- **Cấu trúc cơ bản:**

SELECT [DISTINCT] cot_can_lay_ra	- 5. Chọn những cột nào để hiển thị
FROM ten_bang	- 1. Lấy dữ liệu từ bảng nào
[JOIN ten_bang_khac ON dieu_kien_noi]	- 2. Nối với bảng khác nếu cần
WHERE dieu_kien_loc_hang	- 3. Lọc ra những hàng thỏa mãn điều kiện
GROUP BY cot_de_nhom	- 4. Nhóm các hàng lại
HAVING dieu_kien_loc_nhom	- 6. Lọc các nhóm thỏa mãn điều kiện
ORDER BY cot_de_sap_xep [ASC DESC];	- 7. Sắp xếp kết quả cuối cùng

- **Các toán tử, các hàm được sử dụng khi truy vấn**

- **Một số hàm xử lý chuỗi:** SUBSTRING (), CONCAT (), LEFT (), RIGHT (), UPPER (), LOWER (), REPLACE (), ...
- **Các hàm xử lý số:** ABS (), SIGN (), SQRT (), POWER (), ROUND (), ...
- **Các hàm xử lý ngày:** DAY (), MONTH (), YEAR ()
- **Các hàm tính toán trên nhóm:** MIN (), MAX (), AVG (), SUM (), COUNT ()

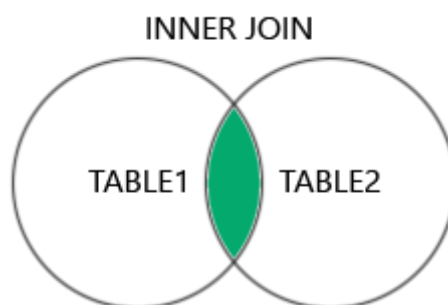
1. FROM và JOIN

- **FROM:** Chỉ định bảng nguồn để lấy dữ liệu.
- **JOIN:** Kết hợp các hàng từ hai hay nhiều bảng dựa trên một cột liên quan. Đây là cách SQL thực thi các phép toán **Cartesian Product** và **Join** trong đại số quan hệ. **Các loại JOIN:**

1.1. INNER JOIN (Phép nối trong)

- Trả về các hàng khi có **sự trùng khớp giá trị ở cả hai bảng**.
- **Cú pháp:**

```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```



- **JOIN** and **INNER JOIN** will return the same result.
- **INNER** is the default join type for **JOIN**, so when you write **JOIN** the parser actually writes **INNER JOIN**.
- **Example:** Consider the two tables, **Student** and **StudentCourse**, which share a common column **ROLL_NO**. Using SQL JOINS, we can combine data from these tables based on their **relationship**, allowing us to retrieve meaningful information like student details along with their **enrolled courses**.

- **Student Table**

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19
6	DHANRAJ	BARABAJAR	XXXXXXXXXX	20
7	ROHIT	BALURGHAT	XXXXXXXXXX	18
8	NIRAJ	ALIPUR	XXXXXXXXXX	19

- **StudentCourse Table**

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

- This query will show the names and age of students enrolled in different courses.

```
SELECT StudentCourse.COURSE_ID, Student.NAME, Student.AGE
FROM Student
INNER JOIN StudentCourse
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

- **Output**

COURSE_ID	NAME	Age
1	HARSH	18
2	PRATIK	19
2	RIYANKA	20
3	DEEP	18
1	SAPTARHI	19

- **JOIN Three Tables:** The following SQL statement selects all orders with customer and shipper information:

```
SELECT Orders.OrderID, Customers.CustomerName, Shippers.ShipperName
FROM (Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID)
INNER JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID);
```

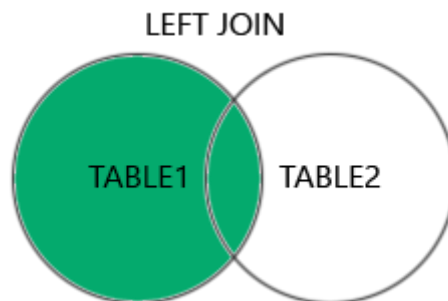
1.2. LEFT JOIN (Phép nối ngoài bên trái)

- Trả về **tất cả các hàng từ bảng bên trái**, và **các hàng trùng khớp từ bảng bên phải**. Nếu không có sự trùng khớp, kết quả là NULL ở các cột của bảng bên phải.

- **Cú pháp:**

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

- **Note:** In some databases LEFT JOIN is called LEFT OUTER JOIN.



- **Example**, the LEFT JOIN retrieves all rows from the Student table and the matching rows from the StudentCourse table based on the ROLL_NO column.

```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
LEFT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

- **Output**

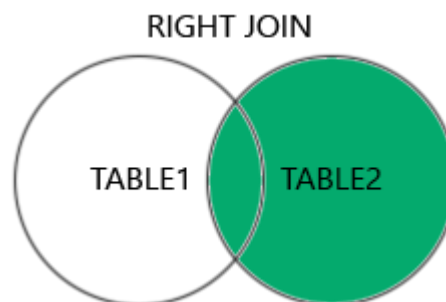
NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
DHANRAJ	NULL
ROHIT	NULL
NIRAJ	NULL

1.3. RIGHT JOIN (Phép nối ngoài bên phải)

- Ngược lại với LEFT JOIN.
- The RIGHT JOIN keyword **returns all records from the right table (table2), and the matching records from the left table (table1).**
- Trả về NULL ở các bản ghi thuộc bảng bên trái nếu không có dữ liệu khớp.
- **Cú pháp:**

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

- **Note:** In some databases RIGHT JOIN is called RIGHT OUTER JOIN.



- **Example**, the RIGHT JOIN retrieves all rows from the **StudentCourse** table and the matching rows from the Student table based on the ROLL_NO column.

```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
RIGHT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

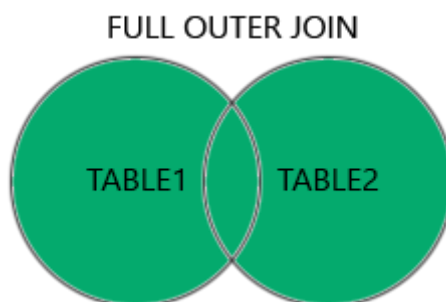
- **Output:**

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
NULL	4
NULL	5
NULL	4

1.4. FULL OUTER JOIN (Phép nối ngoài đầy đủ)

- Trả về tất cả các hàng khi có sự trùng khớp ở **một trong hai bảng**.
- **Tip:** FULL OUTER JOIN and FULL JOIN are the same.
- **Cú pháp:**

```
SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;
```



- **Note:** FULL OUTER JOIN can potentially return very large result-sets!
- This example demonstrates the use of a FULL JOIN, which combines the results of both LEFT JOIN and RIGHT JOIN. The query retrieves all rows from the **Student** and **StudentCourse** tables. If a record in one table does not have a matching record in the other table, the result set will include that record with NULL values for the missing fields

```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
FULL JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

○ **Output**

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
DHANRAJ	NULL
ROHIT	NULL
NIRAJ	NULL
NULL	4
NULL	5
NULL	4

1.5. CROSS JOIN (Tích Đề-các)

- The CROSS JOIN is used to generate a paired combination of each row of the first table with each row of the second table. This join type is also known as cartesian join.
- Trả về tích Đề-các của hai bảng (mỗi hàng của bảng 1 ghép với tất cả các hàng của bảng 2).

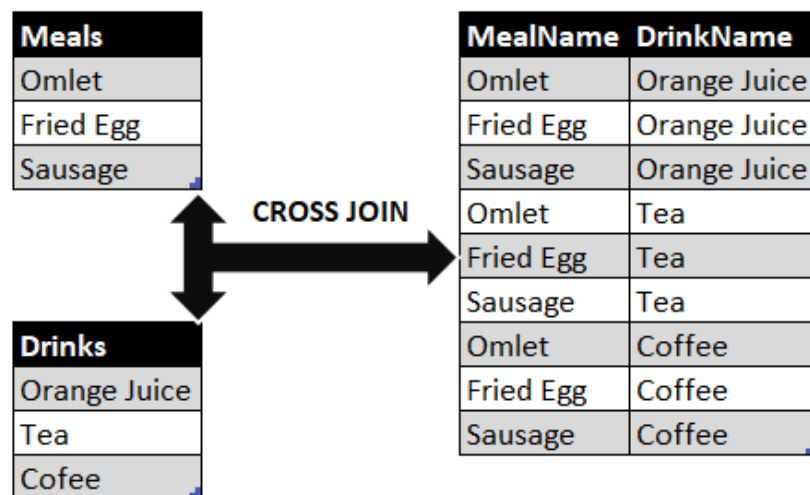
- **Cú pháp:**

```
SELECT ColumnName_1,  
       ColumnName_2,  
       ColumnName_N  
FROM [Table_1]  
CROSS JOIN [Table_2]
```

- Or we can use the following syntax instead of the previous one. This syntax does not include the CROSS JOIN keyword; only we will place the tables that will be joined after the FROM clause and separated with a comma.

```
SELECT ColumnName_1,  
       ColumnName_2,  
       ColumnName_N  
FROM [Table_1],[Table_2]
```

- Unlike the INNER JOIN, LEFT JOIN and FULL OUTER JOIN, the CROSS JOIN does not require a joining condition.
- **Ví dụ:**



```
SELECT * FROM Meals  
CROSS JOIN Drinks
```

1.6. Các phép NATURAL JOIN

- **Difference between Natural join and Inner Join in SQL**

- Natural Join in SQL joins two tables based on the same attribute name and datatypes. The resulting table will contain all the attributes of both tables but keep only one copy of each common column.

- `SELECT * FROM table1 NATURAL JOIN table2;`

- Inner Join joins two table **on the basis of the column which is explicitly specified in the ON clause**. The resulting table will **contain all the attributes from both the tables including common column** also.

- **Example**

Student Table		Marks Table		
Roll_No	Name	Roll_No	Marks	
1	Mohit	2	82	
2	Aman	3	96	
3	Sachin	4	39	
4	Manish	5	98	

SELECT * FROM Student NATURAL JOIN Student_Marks;		Roll_No	Name	Marks
		2	Aman	82
		3	Sachin	96
		4	Manish	39

SELECT * FROM Student S INNER JOIN Student_Marks M ON S.Roll_No = M.Roll_No;		Roll_No	Name	Roll_No	Marks
		2	Aman	2	82
		3	Sachin	3	96
		4	Manish	4	39

- The Natural Joins are **not supported** in the **SQL Server Management Studio** also known as **Microsoft SQL Server**.
- Các phép NATURAL LEFT OUTER JOIN, NATURAL RIGHT OUTER JOIN, NATURAL FULL OUTER JOIN có cùng cơ chế hoạt động như NATURAL JOIN:
 - Các **cột được JOIN tự động phải cùng tên ở cả hai bảng**, và SQL sẽ **ẩn** một trong hai cột đó trong kết quả (tránh trùng lặp).
 - **Không được viết ON ... với NATURAL JOIN** → nếu bạn muốn tùy chỉnh điều kiện, hãy dùng JOIN bình thường.
- **Lưu ý:**
 - **Tránh dùng NATURAL JOIN trong thực tế**, trừ khi bạn **rất chắc** về các cột trùng tên.
 - **Ưu tiên dùng INNER JOIN, LEFT JOIN, v.v.** với điều kiện ON rõ ràng → an toàn, dễ hiểu, dễ bảo trì.

2. WHERE

- Dùng để lọc các bản ghi, chỉ trả về những bản ghi thỏa mãn một điều kiện cụ thể. Đây là cách SQL thực thi phép Selection (σ) trong đại số quan hệ.
- The search condition in the WHERE has the following form:

Left Expression Comparison Operator Right Expression

- Note:** The WHERE clause is not only used in SELECT statements, it is also used in UPDATE, DELETE, etc.!
- Các toán tử phổ biến:**

Loại	Toán tử	Ý nghĩa
Toán tử so sánh Comparison Operators	=	Equal to
	<> or !=	Not equal to
	<	Less than
	>	Greater than
	<=	Less than or equal to
	>=	Greater than or equal to
Toán tử logic Logical Operators	ALL	So sánh với tất cả giá trị trong tập hợp, đúng nếu thỏa mãn với tất cả.
	ANY	So sánh với bất kỳ giá trị nào trong tập hợp, đúng nếu thỏa mãn với ít nhất một giá trị.
	EXISTS	Kiểm tra xem truy vấn con có trả về hàng nào không.
	IN	Kiểm tra giá trị có nằm trong tập hợp không.
	BETWEEN	Kiểm tra giá trị có nằm trong khoảng không.
	LIKE	So khớp chuỗi với một mẫu.
	IS NULL	Kiểm tra giá trị NULL.
	AND	Kết hợp hai điều kiện, đúng nếu cả hai đúng.
	OR	Kết hợp hai điều kiện, đúng nếu ít nhất một đúng.
	NOT	Phủ định điều kiện.

2.1. ALL

- Ý nghĩa:** Toán tử ALL so sánh một giá trị với **tất cả các giá trị** trong một tập hợp (thường là kết quả của một truy vấn con). Kết quả trả về TRUE nếu điều kiện so sánh đúng với **mọi giá trị** trong tập hợp.
- Cú pháp:**

- ALL Syntax With SELECT

```
SELECT ALL column_name(s)
FROM table_name
WHERE condition;
```

- ALL Syntax With WHERE or HAVING

```
SELECT column_name(s)
FROM table_name
WHERE column_name comparison_operator ALL
(SELECT column_name
FROM table_name
WHERE condition);
```

- Nếu tập hợp rỗng, ALL trả về TRUE (theo logic SQL, vì không có giá trị nào làm sai điều kiện).
- Thường được dùng để kiểm tra xem một giá trị có thỏa mãn điều kiện với tất cả các giá trị trong một tập hợp hay không.
- **Ví dụ 1:** Giả sử bạn muốn tìm các sản phẩm có giá lớn hơn **tất cả** giá của sản phẩm trong danh mục 'C01'.

```
SELECT ProductName, UnitPrice
FROM Products
WHERE UnitPrice > ALL
      (SELECT UnitPrice
       FROM Products
       WHERE CategoryID = 'C01');
```

- Truy vấn con (SELECT UnitPrice FROM Products WHERE CategoryID = 'C01') trả về danh sách giá của các sản phẩm trong danh mục 'C01'.
- Điều kiện UnitPrice > ALL (...) chỉ trả về các sản phẩm có UnitPrice lớn hơn **mọi giá trị** trong danh sách đó.
- Nếu danh sách giá trong danh mục 'C01' là {10, 20, 30}, thì chỉ các sản phẩm có giá lớn hơn 30 được trả về.
- **Ví dụ 2:** The following SQL statement lists ALL the product names:

```
SELECT ALL ProductName
FROM Products
WHERE TRUE;
```

- **Lưu ý:**

- **s > ALL R** trả về **TRUE** nếu giá trị vô hướng s **lớn hơn** mọi giá trị trong quan hệ đơn cột R.
→ **ALL** yêu cầu giá trị s phải thỏa mãn điều kiện so sánh với **tất cả** các giá trị trong R.
- Toán tử > có thể được thay thế bằng các toán tử khác như <, >=, <=, = hoặc <>, với ý nghĩa tương ứng:
 - **Ví dụ:**
 - **s = ALL R** trả về **TRUE** nếu s bằng **tất cả** giá trị trong R (thường chỉ đúng khi R chỉ có một giá trị hoặc rỗng).
 - muốn kiểm tra s > ALL R, thì s phải lớn hơn giá trị lớn nhất trong R.
 - **s <> ALL R** tương đương với **s NOT IN R**, nghĩa là s không bằng bất kỳ giá trị nào trong R.
- **NOT s >= ALL R** is **true** if and only if s is not the maximum value in R.

2.2. AND

- Kết hợp hai điều kiện, trả về TRUE **nếu cả hai đều đúng**.
- **Syntax**

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 AND condition2 AND condition3 ...
```

- **Ví dụ:** Tìm khách hàng ở Đức và thành phố Berlin.

```
SELECT * FROM Customers WHERE Country = 'Germany' AND City = 'Berlin';
```

2.3. OR

- Kết hợp hai điều kiện, trả về TRUE nếu **ít nhất một điều kiện đúng**.

- **Syntax**

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

- **Ví dụ:** Tìm khách hàng ở Đức hoặc Pháp.

```
SELECT * FROM Customers WHERE Country = 'Germany' OR Country = 'France';
```

2.4. NOT

- The NOT operator is used in combination with other operators to give the opposite result, also called the negative result.
- Phủ định một điều kiện, trả về TRUE nếu điều kiện là sai.

- **Syntax**

```
SELECT column1, column2, ...  
FROM table_name  
WHERE NOT condition;
```

- **Ví dụ:** Tìm khách hàng không ở Đức.

```
SELECT * FROM Customers WHERE NOT Country = 'Germany';
```

2.5. IN

- Kiểm tra xem giá trị có nằm trong một tập hợp không.
- **Tương đương với nhiều điều kiện OR.**

- **Syntax**

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name IN (value1, value2, ...);  
Hoặc  
WHERE column_name IN (subquery);
```

- **Ví dụ 1:** Tìm khách hàng ở Paris hoặc London.

```
SELECT * FROM Customers WHERE City IN ('Paris', 'London');
```

Tương đương:

```
SELECT * FROM Customers WHERE City = 'Paris' OR City = 'London';
```

- **Ví dụ 2:** Return all customers that have an order in the **Orders** table:

```
SELECT * FROM Customers  
WHERE CustomerID IN (SELECT CustomerID FROM Orders);
```

- **NOT IN:** By using the NOT keyword in front of the IN operator, you return all records that are NOT any of the values in the list.

Ví dụ 3: Return all customers that are NOT from 'Germany', 'France', or 'UK':

```
SELECT * FROM Customers  
WHERE Country NOT IN ('Germany', 'France', 'UK');
```

2.6. BETWEEN

- Kiểm tra xem giá trị có nằm trong một khoảng giá trị nào đó không (bao gồm cả hai đầu mút).
- The values can be numbers, text, or dates.
- The **BETWEEN** operator is inclusive: **begin** and **end values** are included.
- **Syntax**

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

- To display the products outside the range of the previous example, use **NOT BETWEEN**.
- **Ví dụ 1:** Tìm sản phẩm có giá từ 10 đến 20.

```
SELECT * FROM Products WHERE Price BETWEEN 10 AND 20;
```

Tương đương:

```
SELECT * FROM Products WHERE Price >= 10 AND Price <= 20;
```

- **Ví dụ 2: BETWEEN Text Values:** The following SQL statement selects all products with a ProductName **alphabetically** between Carnarvon Tigers and Mozzarella di Giovanni:

```
SELECT * FROM Products
WHERE ProductName BETWEEN 'Carnarvon Tigers' AND 'Mozzarella di Giovanni'
ORDER BY ProductName;
```

- **Ví dụ 3: BETWEEN Dates:** The following SQL statement selects all orders with an OrderDate between '01-July-1996' and '31-July-1996':

```
SELECT * FROM Orders
WHERE OrderDate BETWEEN #07/01/1996# AND #07/31/1996#;
```

OR:

```
SELECT * FROM Orders
WHERE OrderDate BETWEEN '1996-07-01' AND '1996-07-31';
```

- **Ví dụ 4: BETWEEN with IN:** The following SQL statement selects all products with a price between 10 and 20. In addition, the CategoryID must be either 1, 2, or 3:

```
SELECT * FROM Products
WHERE Price BETWEEN 10 AND 20
AND CategoryID IN (1,2,3);
```

2.7. LIKE

- So khớp chuỗi với một mẫu, sử dụng ký tự đại diện (% cho chuỗi bất kỳ, _ cho một ký tự).
- **Wildcard Characters**

Symbol	Description
%	Represents <u>zero, one, or multiple</u> characters
_	Represents <u>one, single</u> character
[]	Represents any single character within the brackets *
^	Represents any character not in the brackets *
-	Represents any single character within the specified range *
{}	Represents any escaped character **

* Not supported in PostgreSQL and MySQL databases.

** Supported only in Oracle databases.

- **Syntax**

```
SELECT column1, column2, ...  
FROM table_name  
WHERE columnN LIKE pattern;
```

- **Example**

- Select all customers that **starts** with the letter "a":

```
SELECT * FROM Customers  
WHERE CustomerName LIKE 'a%';
```

- Return all customers from a city that starts with 'L' followed by one wildcard character, then 'nd' and then two wildcard characters:

```
SELECT * FROM Customers  
WHERE city LIKE 'L_nd__';
```

- Return all customers that **starts** with 'La':

```
SELECT * FROM Customers  
WHERE CustomerName LIKE 'La%';
```

- Return all customers that **starts** with 'a' **or starts** with 'b':

```
SELECT * FROM Customers  
WHERE CustomerName LIKE 'a%' OR CustomerName LIKE 'b%';
```

- Return all customers that **ends** with 'a':

```
SELECT * FROM Customers  
WHERE CustomerName LIKE '%a';
```

- Return all customers that **starts** with "b" and **ends** with "s":

```
SELECT * FROM Customers  
WHERE CustomerName LIKE 'b%s';
```

- Return all customers from a city that **contains** the letter 'L':

```
SELECT * FROM Customers  
WHERE city LIKE '%L%';
```

- Return all customers that **contains the phrase** 'or'

```
SELECT * FROM Customers  
WHERE CustomerName LIKE '%or%';
```

- Return all customers that **starts with** "a" and are **at least 3 characters in length**:

```
SELECT * FROM Customers  
WHERE CustomerName LIKE 'a__%';
```

- Return all customers that have "r" in the **second position**:

```
SELECT * FROM Customers  
WHERE CustomerName LIKE '_r%';
```

- Return all customers from Spain:

```
SELECT * FROM Customers  
WHERE Country LIKE 'Spain';
```

2.8. IS NULL/ IS NOT NULL

- Kiểm tra giá trị NULL (giá trị không tồn tại hoặc chưa biết).
- **What is a NULL Value?**
 - A field with a NULL value is **a field with no value.**
 - If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value.
 - **Note:** A NULL value is different from a zero value or a field that contains spaces. A field with a NULL value is one that has been left blank during record creation!
- **IS NULL Syntax**

```
SELECT column_names
FROM table_name
WHERE column_name IS NULL;
```

- **IS NOT NULL Syntax**

```
SELECT column_names
FROM table_name
WHERE column_name IS NOT NULL;
```

- **Ví dụ:**
 - Tìm khách hàng không có địa chỉ:

```
SELECT * FROM Customers WHERE Address IS NULL;
```
 - Tìm khách hàng có địa chỉ:

```
SELECT * FROM Customers WHERE Address IS NOT NULL;
```
- **Lưu ý về NULL:** Các phép so sánh (=, <>, v.v.) với NULL luôn trả về FALSE. Do đó, phải dùng IS NULL hoặc IS NOT NULL.

2.9. ANY

- Toán tử ANY so sánh một giá trị với **bất kỳ giá trị nào** trong một tập hợp (thường là kết quả của một truy vấn con).
- Kết quả trả về TRUE nếu điều kiện so sánh đúng với ít nhất một giá trị trong tập hợp.
- **Cú pháp:**

```
SELECT column_name(s)
FROM table_name
WHERE column_name comparison_operator ANY
(SELECT column_name
FROM table_name
WHERE condition);
```

- Tương tự như IN khi sử dụng với =, nhưng ANY linh hoạt hơn vì có thể dùng với các toán tử so sánh khác (>, <, v.v.).
- Nếu tập hợp rỗng, ANY trả về FALSE.
- **Lưu ý:**
 - **s > ANY R** trả về **TRUE** nếu giá trị vô hướng s **lớn hơn** ít nhất một giá trị trong quan hệ đơn cột R.
→ **ANY** yêu cầu giá trị s chỉ cần thỏa mãn điều kiện so sánh với **ít nhất một** giá trị trong R.

- Toán tử > có thể được thay thế bằng các toán tử khác như <, >=, <=, = hoặc <>, với ý nghĩa tương ứng:
 - Ví dụ:
 - **s = ANY R** tương đương với **s IN R**, nghĩa là s bằng ít nhất một giá trị trong R.
 - muốn kiểm tra s > ANY R, thì s chỉ cần lớn hơn ít nhất một giá trị trong R (thường là giá trị nhỏ nhất).
 - **s <> ANY R** trả về **TRUE** nếu s khác ít nhất một giá trị trong R.
 - **NOT s > ANY R** is **true** if and only if s is the minimum value in R.
- **Ví dụ:** Tìm các sản phẩm có giá lớn hơn **ít nhất một** giá của sản phẩm trong danh mục 'C01'.

```
SELECT ProductName, UnitPrice
FROM Products
WHERE UnitPrice > ANY
      ( SELECT UnitPrice
        FROM Products
        WHERE CategoryID = 'C01' );
```

- Truy vấn con trả về danh sách giá của các sản phẩm trong danh mục 'C01', ví dụ: {10, 20, 30}.
- Điều kiện UnitPrice > ANY (...) trả về các sản phẩm có UnitPrice lớn hơn **ít nhất một giá trị** trong danh sách (tức là lớn hơn 10).
- Kết quả sẽ bao gồm các sản phẩm có giá lớn hơn 10 (như 15, 20, 30, 35, v.v.).
- **Lưu ý:** Tương tự ALL, trong SQLite, ANY có thể không được hỗ trợ trực tiếp. Bạn có thể thay thế bằng MIN hoặc MAX. Ví dụ:

```
SELECT ProductName, UnitPrice
FROM Products
WHERE UnitPrice >
      ( SELECT MIN(UnitPrice)
        FROM Products
        WHERE CategoryID = 'C01' );
```

Truy vấn này trả về các sản phẩm có giá lớn hơn giá nhỏ nhất trong danh mục 'C01'.

- **So sánh với IN:**
 - UnitPrice **=** ANY (subquery) tương đương với UnitPrice **IN** (subquery).
 - UnitPrice **>** ANY (subquery) tương đương với UnitPrice > **(SELECT MIN(UnitPrice) FROM ...)**.

2.10. EXISTS

- Toán tử EXISTS kiểm tra xem một truy vấn con (một quan hệ R) có trả về bất kỳ hàng nào hay không.
- Nếu truy vấn con **trả về ít nhất một hàng**, EXISTS trả về **TRUE**; nếu không, trả về **FALSE**.
- **Cú pháp:**

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS
      (SELECT column_name FROM table_name WHERE condition);
```

- Thường được dùng để **kiểm tra sự tồn tại của dữ liệu trong một bảng** liên quan.
- Hiệu quả hơn IN trong một số trường hợp, đặc biệt khi truy vấn con trả về nhiều hàng, vì EXISTS dừng lại ngay khi tìm thấy một hàng khớp.

- **NOT EXISTS R** trả về **TRUE** nếu quan hệ R rỗng, tức là không có dòng nào trong R.
- **Ví dụ 1:** Tìm tất cả khách hàng có ít nhất một đơn hàng.

```
SELECT * FROM Customers
WHERE EXISTS
    ( SELECT * FROM Orders
      WHERE Orders.CustomerID = Customers.CustomerID );
```

- Truy vấn con kiểm tra xem có tồn tại bất kỳ đơn hàng nào trong bảng Orders có CustomerID khớp với CustomerID của bảng Customers.
- Nếu truy vấn con trả về ít nhất một hàng, EXISTS trả về TRUE, và khách hàng đó được bao gồm trong kết quả.
- Nếu không có đơn hàng nào khớp, EXISTS trả về FALSE, và khách hàng đó bị loại khỏi kết quả.
- **Ví dụ 2:** Tìm các sản phẩm không có trong bất kỳ đơn hàng nào.

```
SELECT ProductName FROM Products
WHERE NOT EXISTS
    ( SELECT * FROM OrderDetails
      WHERE OrderDetails.ProductID = Products.ProductID );
```

- NOT EXISTS trả về TRUE nếu truy vấn con không trả về hàng nào, tức là sản phẩm không xuất hiện trong bảng OrderDetails.
- Kết quả là danh sách các sản phẩm chưa từng được đặt hàng.
- **Lưu ý:**
 - EXISTS thường hiệu quả hơn IN khi làm việc với tập dữ liệu lớn, vì nó không cần tải toàn bộ kết quả của truy vấn con vào bộ nhớ.
 - EXISTS chỉ quan tâm đến **sự tồn tại của hàng**, không quan tâm đến giá trị cụ thể.

3. GROUP BY

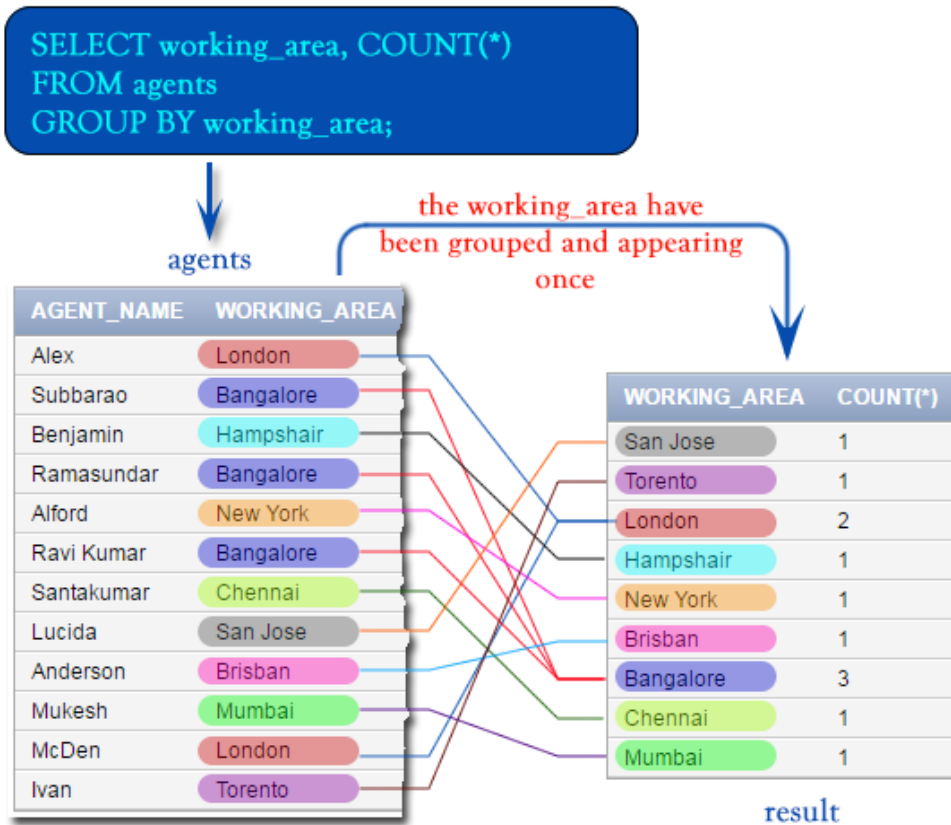
- Nhóm các hàng có cùng giá trị trong các cột được chỉ định thành các hàng tóm tắt.
- GROUP BY thường được sử dụng với các hàm tổng hợp (COUNT, MAX, MIN, SUM, AVG) để thực hiện tính toán trên mỗi nhóm.

```
SELECT < One or more Attributes in Group clause > [, Aggregation Functions]
FROM < Relations >
[WHERE < Tuple-Condition >]
GROUP BY <Attribute want to Group>
[HAVING < Group-Condition >]
```

- **Các hàm tổng hợp (Aggregate Functions)**

Hàm	Mô tả	Tham số (Argument) (.): Là một biểu thức trả về giá trị vô hướng (scalar), thường là tên cột . Ví dụ: SELECT Sum(salary) FROM Employee; Count(*): Counts all the tuples in the relation that agree with condition on WHERE clause, kể cả dòng có NULL . → Tất cả các hàm tổng hợp (trừ COUNT(*)) đều bỏ qua giá trị NULL.
COUNT(.)	Đếm số lượng hàng	
SUM(.)	Tính tổng các giá trị	
AVG(.)	Tính giá trị trung bình	
MIN(.)	Tìm giá trị nhỏ nhất	
MAX(.)	Tìm giá trị lớn nhất	

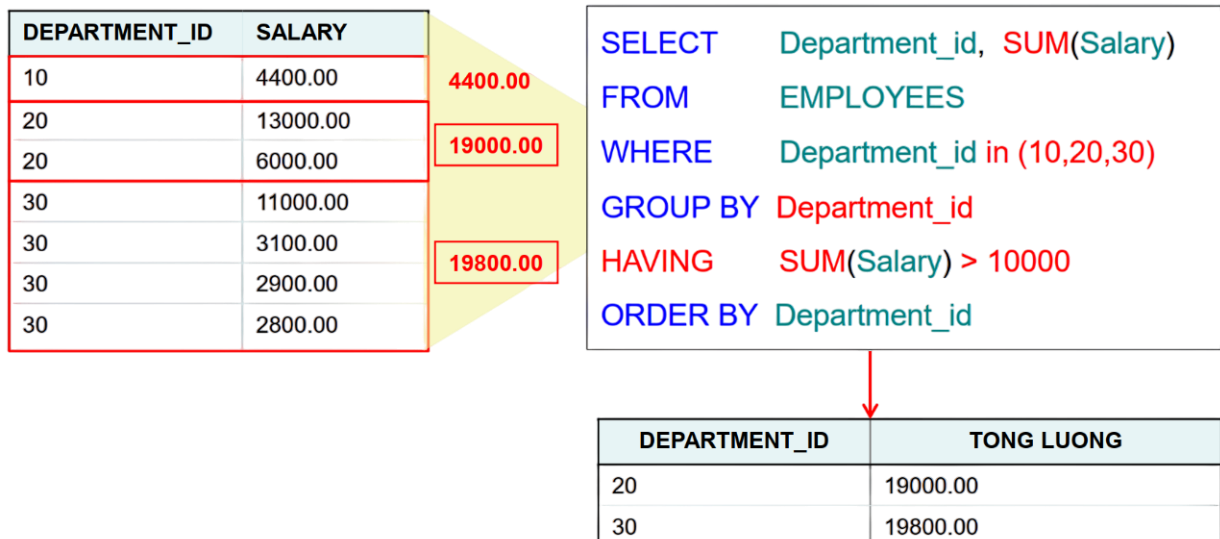
- **Ví dụ 1:**



- **Ví dụ 2:** Đếm số đơn hàng của mỗi khách hàng.

```
SELECT CustomerID, COUNT(OrderID) AS NumberOfOrders
FROM Orders
GROUP BY CustomerID;
```

- **Ví dụ 3:**



- SQL system does not eliminate duplicates. Thus, the SQL response to a query may list the same tuple several times. If we do not wish duplicates in the result, then we may follow the keyword **SELECT** by the keyword **DISTINCT**.

SELECT DISTINCT X

- **DISTINCT** dùng để loại bỏ các dòng trùng lặp trong kết quả của câu truy vấn, đảm bảo mỗi dòng trong kết quả chỉ xuất hiện một lần.
- **COUNT(DISTINCT column_name):** count the number of **unique, non-null values** within a specified column.

ID	A	Khi đó: - Các giá trị trong A là: 10, 20, 10, NULL, 30, 20 - Các giá trị khác nhau (distinct) là: 10, 20, 30 → COUNT(DISTINCT A) = 3
1	10	
2	20	
3	10	
4	NULL	
5	30	
6	20	

- **Ví dụ 4: Cho bảng** SalesLT.Product chứa thông tin các sản phẩm:

ProductID	ProductModelID	ProductCategoryID	ListPrice
1	101	5	1000.00
2	102	5	1200.00
3	101	5	1500.00
4	103	6	2000.00
5	104	6	NULL
6	105	6	2500.00

- **Phân tích thông tin giá cả sản phẩm theo từng loại hàng hóa (ProductCategoryID),** gồm: Tổng giá, Giá trung bình, Số lượng sản phẩm, Số mô hình sản phẩm khác nhau, Giá cao nhất, Giá thấp nhất.

```

SELECT
    ProductCategoryID,
    SUM(ListPrice) AS sum_listprice,
    AVG(ListPrice) AS avg_listprice,
    COUNT(ProductID) AS count_productid,
    COUNT(DISTINCT ProductModelID) AS count_distinct_productmodelid,
    MAX(ListPrice) AS max_listprice,
    MIN(ListPrice) AS min_listprice
FROM SalesLT.Product
GROUP BY ProductCategoryID

```

- **Giải thích:**
 - **SELECT ProductCategoryID,** → Hiển thị nhóm sản phẩm theo ProductCategoryID (nhóm phân loại sản phẩm).
 - **SUM(ListPrice) AS sum_listprice,** → Tính **tổng giá sản phẩm** (chỉ tính các dòng có ListPrice ≠ NULL).
 - **AVG(ListPrice) AS avg_listprice,** → Tính **giá trung bình** của sản phẩm trong mỗi nhóm (bỏ qua NULL).
 - **COUNT(ProductID) AS count_productid,** → Đếm **số sản phẩm** trong mỗi nhóm (ProductID luôn có, nên tính cả dòng có ListPrice = NULL).
 - **COUNT(DISTINCT ProductModelID) AS count_distinct_productmodelid,** → Đếm **số lượng model khác nhau** trong nhóm sản phẩm.
 - **MAX(ListPrice) AS max_listprice, MIN(ListPrice) AS min_listprice** → Tìm giá cao nhất và thấp nhất trong nhóm (bỏ NULL).
 - **FROM SalesLT.Product GROUP BY ProductCategoryID;** → Gom các dòng lại theo ProductCategoryID, sau đó tính các thống kê như trên cho từng nhóm.

- **Kết quả:**

Product CategoryID	sum_listprice	avg_listprice	count_productid	count_distinct_productmodelid	max_listprice	min_listprice
5	3700.00	1233.33	3	2	1500.00	1000.00
6	4500.00	2250.00	3	3	2500.00	2000.00

(Lưu ý: NULL bị bỏ qua trong SUM, AVG, MAX, MIN)

- **Chú ý:**

- SQL coi **NULL là một giá trị bình thường khi GROUP BY**. Tức là: nếu một số dòng có NULL ở cột dùng để GROUP BY, thì **chúng sẽ được gom thành một nhóm riêng biệt**.

Dept	Salary	SELECT Dept, COUNT(*) AS num_people FROM Employees GROUP BY Dept;	Kết quả:	
IT	6000		Dept	num_people
IT	7000		IT	2
NULL	5000		NULL	2
NULL	4000		Sales	1
Sales	8000			

- Khi bạn **gom nhóm** mà không có dữ liệu để gom (do lọc dữ liệu chẳng hạn):
 - **SUM, AVG, MIN, MAX trên tập rỗng → trả về NULL**
 - **COUNT(*) hoặc COUNT(A) trên tập rỗng → trả về 0**

4. HAVING

- Dùng để **lọc các nhóm sau khi đã được tạo bởi GROUP BY**.

Criteria	WHERE Clause	HAVING Clause
<i>Purpose</i>	HAVING lọc hàng sau khi gom nhóm (GROUP BY).	WHERE lọc hàng trước khi gom nhóm.
<i>Used With</i>	Works with individual rows.	Works with grouped or aggregated data.
<i>Applicable To</i>	Columns in the table (individual records).	Aggregated results (e.g., SUM(), COUNT()).
<i>Aggregate Functions</i>	HAVING clause can only be used with aggregate function.	The WHERE Clause cannot be used with aggregate function like Having
<i>Stage of Query Execution</i>	Priority Wise HAVING Clause is executed after Group By.	Priority Wise WHERE is executed before Group By.
<i>Order in Query</i>	Appears before GROUP BY.	Appears after GROUP BY.

- **There are several rules we must remember about HAVING clauses:**

- "An aggregation in a HAVING clause applies only to the tuples of the group being tested". Nghĩa là: Khi bạn viết HAVING SUM(price) > 100, thì **mỗi nhóm** sẽ được tính SUM(price) riêng, rồi mới so sánh > 100.
- "Any attribute of relations in the FROM clause may be aggregated in the HAVING clause". Nghĩa là: Bạn có thể áp dụng các hàm tổng hợp (SUM, COUNT, AVG, ...) lên **bất kỳ cột nào từ bảng trong FROM**, miễn là hợp lệ.

- **Ví dụ:** Chỉ hiển thị những khách hàng có nhiều hơn 5 đơn hàng.

```
SELECT CustomerID, COUNT(OrderID) AS NumberOfOrders
FROM Orders
GROUP BY CustomerID
HAVING COUNT(OrderID) > 5;
```

- **Example 1: Using WHERE Clause for Row-Level Filtering:** Consider the following table **Marks**, which contains **student names**, their respective **courses**, and the scores they achieved. This example demonstrates how the **WHERE clause** can filter data at the **individual row level**, before any **grouping** or **aggregation** takes place.

Student	Course	Score
a	c1	40
a	c2	50
b	c3	60
d	c1	70
e	c2	80

- To select students with a score greater than or equal to 40, we can use the **WHERE** clause. This query filters rows before any aggregation, selecting only students with scores **greater than or equal to 40**.

SELECT Student, Score FROM Marks WHERE Score >= 40;	Student	Score
	a	40
	a	50
	b	60
	d	70
	e	80

- **Example 2: Using HAVING Clause for Aggregated Data Filtering:** Suppose you want to calculate the **total score of each student** and only show students whose **total score is greater than 70**. We can use the **HAVING** clause:

SELECT Student, SUM(Score) AS Total FROM Marks GROUP BY Student HAVING Total > 70;	Student	Total
	a	90
	e	80

- **Explanation:** Here, we are using **HAVING** to filter groups of students after summing their scores. This is because **HAVING** works on aggregated data, whereas **WHERE** cannot be used for filtering after aggregation.

5. SELECT

- Chỉ định các cột bạn muốn trả về trong kết quả. Đây là **cách SQL thực thi phép Projection (π)** trong đại số quan hệ.
 - **SELECT *:** Chọn tất cả các cột.
 - **SELECT DISTINCT cot:** Trả về các giá trị duy nhất (không trùng lặp).

6. ORDER BY

- Sắp xếp tập kết quả theo một hoặc nhiều cột.

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1, column2, ... ASC|DESC;
```

- ASC: Tăng dần (**mặc định**).
- DESC: Giảm dần.

- Ví dụ:**

- Sort the products by price:

```
SELECT * FROM Products  
ORDER BY Price;
```

- Sort the products from highest to lowest price:

```
SELECT * FROM Products  
ORDER BY Price DESC;
```

- The following SQL statement selects all customers from the "Customers" table, sorted ascending by the "Country" and descending by the "CustomerName" column:

```
SELECT * FROM Customers  
ORDER BY Country ASC, CustomerName DESC;
```

- The following SQL statement selects all customers from the "Customers" table, sorted by the "Country" and the "CustomerName" column. This means that it orders by Country, but if some rows have the same Country, it orders them by CustomerName:

```
SELECT * FROM Customers  
ORDER BY Country, CustomerName;
```

- Sort the products alphabetically by ProductName:

```
SELECT * FROM Products  
ORDER BY ProductName;
```

- Sort the products by ProductName in reverse order:

```
SELECT * FROM Products  
ORDER BY ProductName DESC;
```

7. Subqueries (Truy vấn con)

- Là một câu lệnh SELECT lồng bên trong một câu lệnh SQL khác.
- Nó có thể nằm trong: WHERE, FROM, SELECT, HAVING, v.v.
- Subquery có thể:
 - Trả về **một giá trị** (scalar/ single constant) → this constant can be compared with another value in a WHERE clause.
 - Trả về **một tập giá trị** (list)
 - Trả về **một bảng** (relation)
- Subqueries can have subqueries, and so on, down as many levels as we wish.
- Subqueries can appear in FROM clauses.
 - Subqueries is as relations in a FROM clause.**
 - In a FROM list, use a parenthesized subquery and give it a alias.
 - E.g.** Finding information about product that store in warehouse with WarehouseID = 'W01'.

```

SELECT A.*
FROM Products A, ( SELECT ProductID
                    FROM Instocks
                    WHERE WarehouseID = 'W01'
                  ) B
WHERE A.ProductID = B.ProductID;

```

- **Ví dụ:** Tìm các sản phẩm được đặt trong đơn hàng 'D01'.

```

SELECT ProductName
FROM Products
WHERE ProductID IN
    (SELECT ProductID
     FROM OrderDetails
     WHERE OrderID = 'D01');

```

- Tính tổng tiền của từng khách hàng đã đặt hàng

```

SELECT CustomerID, SUM(TotalAmount) AS Total
FROM (
    SELECT CustomerID, Amount AS TotalAmount
    FROM Orders
) AS OrderSummary
GROUP BY CustomerID;

```

- Truy vấn con trong FROM tạo bảng tạm OrderSummary
- Truy vấn chính tính tổng tiền theo CustomerID

8. Các phép toán khác

8.1. Union U

- The **UNION** operator is used to combine the result-set of two or more **SELECT** statements.
 - Every **SELECT** statement within **UNION** must have **the same number of columns**
 - The columns must also have **similar data types**
 - The columns in every **SELECT** statement must also be **in the same order**

- **Syntax**

```

SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;

```

- The **UNION** operator **selects only distinct values by default**. To **allow duplicate values**, use **UNION ALL**:

```

SELECT column_name(s) FROM table1
UNION ALL
SELECT column_name(s) FROM table2;

```

- **Note:** The **column names in the result-set** are usually **equal to the column names in the first SELECT statement**.

- **So sánh UNION và UNION ALL**

UNION	UNION ALL
Nó chỉ trả về các hàng riêng biệt từ tập kết quả của hai truy vấn → Loại bỏ các giá trị trùng lặp, KQ "sạch" hơn. → Thêm tác vụ xử lý → Tốc độ truy vấn chậm hơn, tiêu tốn nhiều bộ nhớ & tài nguyên CPU hơn → Ảnh hưởng hiệu suất.	Nó trả về những giá trị trùng lặp từ tập kết quả của hai truy vấn. → Không loại bỏ trùng lặp → Hiệu suất tốt hơn. → Tập hợp đầy đủ các bản ghi. → Tập kết quả lớn.
Chậm hơn so với toán tử UNION ALL .	Triển khai nhanh vì không cần bộ lọc tập kết quả bằng cách loại bỏ các giá trị trùng lặp.

- **Ví dụ:** Let's create two tables "Emp1" and "Emp2".

- **Emp1 Table**

EmpID	Name	Country	Age	mob
1	Shubham	India	23	738479734
2	Aman	Australia	21	436789555
3	Naveen	Sri lanka	24	34873847
4	Aditya	Austria	21	328440934
5	Nishant	Spain	22	73248679

- **Emp2 Table**

EmpID	Name	Country	Age	mob
1	Tommy	England	23	738985734
2	Allen	France	21	43678055
3	Nancy	India	24	34873847
4	Adi	Ireland	21	320254934
5	Sandy	Spain	22	70248679

- **Example 1: SQL UNION Operator:** In this example, we will find the cities (only unique values) from both the "Table1" and the "Table2" tables:

SELECT Country FROM Emp1 UNION SELECT Country FROM Emp2 ORDER BY Country;		Country
		Australia
		Austria
		England
		France
		India
		Ireland
		Spain
		Sri lanka

- **Example 2: SQL UNION ALL:** In the below example, we will find the cities (duplicate values also) from both the "Emp1" and the "Emp2" tables:

SELECT Country FROM Emp1 UNION ALL SELECT Country FROM Emp2 ORDER BY Country;		Country	
		Australia	
		Austria	
		England	
		France	
		India	
		India	
		Ireland	
		Spain	
		Spain	
		Sri lanka	

- **Example 3: SQL UNION ALL With WHERE:** You can use the WHERE clause with UNION ALL in SQL. The WHERE clause is used to filter records and is added after each SELECT statement
 - The following SQL statement returns the cities (duplicate values also) from both the "Geeks1" and the "Geeks2" tables:

SELECT Country, Name FROM Emp1 WHERE Name='Aditya' UNION ALL SELECT Country, Name FROM Emp2 WHERE Country='Ireland' ORDER BY Country;	<table> <tr> <th>Country</th><th>Name</th></tr> <tr> <td>Austria</td><td>Aditya</td></tr> <tr> <td>Ireland</td><td>Adi</td></tr> </table>	Country	Name	Austria	Aditya	Ireland	Adi
Country	Name						
Austria	Aditya						
Ireland	Adi						

- **UNION và FULL OUTER JOIN**

Tiêu chí	UNION	FULL OUTER JOIN
Mục đích chính	Gộp các dòng từ hai tập hợp (bảng)	Kết hợp các dòng từ hai bảng dựa trên điều kiện
Cách hoạt động	Ghép các dòng theo chiều đọc	Ghép các dòng theo điều kiện JOIN (thường là key)
Cột yêu cầu	Hai bảng phải có số lượng & kiểu cột giống nhau	Hai bảng không cần giống nhau hoàn toàn
Ghép theo điều kiện?	Không	Có (ON hoặc USING)
Kết quả trùng lặp	Loại bỏ trùng lặp mặc định (UNION)	Không loại bỏ gì cả (có thể có NULL nếu không khớp)
NULL ở đâu?	Không có NULL thêm vào	Có thể có NULL ở bên trái/phải nếu không khớp

8.2. Intersection $\cap$

- It returns only the rows that are present in both result sets.
- Requires that both SELECT statements have the same number of columns.
- The data types of corresponding columns (các cột tương ứng) in both queries must be compatible (tương thích ~ giống nhau).
- **Syntax:**

```
SELECT column1 , column2 ....
```

```
FROM table1
```

```
WHERE condition
```

INTERSECT

```
SELECT column1 , column2 ....
```

```
FROM table2
```

```
WHERE condition
```

- INTERSECT can be slower on large datasets as it performs row-by-row comparison. Indexing can help optimize performance.
- INTERSECT treats NULL values as **equal**, meaning if both queries return a row with NULL, it will be included in the result.
- **Alternative Approach:** In cases where INTERSECT is not supported (e.g., MySQL), you can achieve the same result using INNER JOIN.
- **Example:** Let's consider two tables: the **Customers** table, which holds **customer details**, and the **Orders** table, which contains information about customer purchases. By applying the **INTERSECT** operator, we can retrieve customers who exist in both tables, meaning those who have made purchases.

- **Customers Table**

CustomerID	FirstName	LastName	Email	City
1	John	Doe	john.doe@example.com	New York
2	Jane	Smith	jane.smith@example.com	Los Angeles
3	Michael	Brown	michael.brown@example.com	Chicago
4	Sarah	Johnson	sarah.johnson@example.com	Houston
5	Emily	Davis	emily.davis@example.com	Phoenix
6	David	Wilson	david.wilson@example.com	Philadelphia
7	Sophia	Anderson	sophia.anderson@example.com	San Antonio
8	James	Taylor	james.taylor@example.com	San Diego

- **Orders Table**

OrderID	CustomerID	Product	Amount	OrderDate
101	2	Laptop	1200	2024-01-10
102	3	Phone	800	2024-01-12
103	5	Tablet	600	2024-01-15
104	6	Headphones	150	2024-01-18
105	7	Smartwatch	300	2024-01-20
106	8	Monitor	250	2024-01-22
107	9	Keyboard	100	2024-01-25
108	10	Mouse	50	2024-01-28

- **Example 1:** we retrieve customers who exist in both the **Customers** and **Orders** tables. The INTERSECT operator ensures that only those customers who have placed an order appear in the result.

SELECT CustomerID FROM Customers INTERSECT SELECT CustomerID FROM Orders;		CustomerID	
		2	
		3	
		5	
		6	
		7	
		8	

- The query returns only those customers who appear in both the Customers and Orders tables.
- If a customer exists in Customers but has never placed an order, they won't appear in the result.
- Customer IDs **2, 3, 5, 6, 7, and 8** appear in both the Customers and Orders tables
- **Example 2: Using INTERSECT with BETWEEN Operator:** we apply the INTERSECT operator along with the BETWEEN condition to filter records based on a specified range. The query retrieves customers whose **CustomerID** falls between **3 and 8** and who have placed an order. The result contains only the common **CustomerID values** that meet both conditions.

SELECT CustomerID FROM Customers WHERE CustomerID BETWEEN 3 AND 8 INTERSECT SELECT CustomerID FROM Orders;		CustomerID	
		3	
		5	
		6	
		7	
		8	

- The first SELECT statement filters customers with CustomerID **between 3 and 8**.
- The INTERSECT operator ensures that only customers from this filtered set who have placed an order are included in the result.
- **Customers 3, 5, 6, 7, and 8** fall within the specified range (**3 to 8**).

8.3. Difference –

- The SQL EXCEPT operator is used to return the rows from the **first SELECT statement** that are not present in the **second SELECT statement**.
- This operator is conceptually similar to the subtract operator in relational algebra.
- It is useful for finding records in one table that do not have corresponding records in another table.
- Use EXCEPT when you need to exclude certain rows efficiently from the first result set, especially in larger datasets.

- Syntax:**

```
SELECT column_name(s)
FROM table1
    EXCEPT
SELECT column_name(s)
FROM table2;
```

- Note:** The two SELECT queries must return the same number of columns and the data types must be compatible.
- EXCEPT automatically removes duplicates, while **EXCEPT ALL** retains duplicates.
- EXCEPT is not supported in MySQL, while NOT IN can be used as an alternative.
- SQL EXCEPT vs. SQL NOT IN:** While both **EXCEPT** and **NOT IN** are used to exclude certain records, there are important differences between them.

Feature	EXCEPT	NOT IN
Duplicate Handling	Removes duplicates from the result	Retains duplicates in the result
Performance	Generally more efficient for large datasets as it processes only the required rows	May be slower for large datasets, especially when checking multiple conditions
Use Case	When you need to find rows that exist in one result set but not the other	When you need to check a specific column's values against a list
SQL Support	Not supported by MySQL	Supported by most SQL databases

- Examples:** Let's consider two tables, **Students** and **Teaching Assistant**.

Students Table				Teaching Assistant Table			
	StudentID	Name	Course		StudentID	Name	Course
	1	Rohan	DBMS		1	Kevin	TOC
	2	Kevin	OS		2	Sita	IP
	3	Mansi	DBMS		3	Manik	AP
	4	Mansi	ADA		4	Rekha	SNS
	5	Rekha	ADA				
	6	Megha	OS				

- Example 1: Filter Student:** We want to find all the students who are not teaching assistants.

SELECT Name FROM Students EXCEPT SELECT Name FROM TA;		Name Mansi Megha Rohan	
---	--	--	--

- The **EXCEPT operator** returns the names that are present in the Students table but not in the TA table. Notice that "Rohan", "Mansi", and "Megha" are returned, as these students are not listed in the TA table.

- **Example 2: Retaining Duplicates with EXCEPTALL:** By default, **EXCEPT** removes duplicates from the result set. To retain duplicates, you can use **EXCEPT ALL** instead.

SELECT Name FROM Students EXCEPT ALL SELECT Name FROM TA;	Name	
	Rohan	
	Mansi	
	Mansi	
	Megha	

- In this case, "Mansi" appears twice in the output because it appears twice in the Students table and is not in the TA table. EXCEPT ALL retains duplicates from the first result set.

9. Conditions Involving Tuples (Điều kiện liên quan đến các bộ dữ liệu)

- Trong SQL, **khi một tuple có số lượng thành phần bằng với số cột của một quan hệ (relation) R**, ta có thể sử dụng các toán tử so sánh như **IN, = ANY, <> ANY, v.v.** để so sánh tuple với các dòng trong R.
- Ví dụ: (a, b) IN R kiểm tra xem cặp giá trị (a, b) có tồn tại trong quan hệ R hay không, tức là có một dòng trong R có giá trị các cột tương ứng khớp với (a, b).
- Quan hệ R thường là kết quả của một subquery (truy vấn con) trả về một tập hợp các dòng với số cột khớp với tuple.
- **Ví dụ lược đồ cơ sở dữ liệu Warehouse Management:**
 - **Categories(CategoryID, CategoryName, Description):** Lưu thông tin danh mục sản phẩm.
 - **Products(ProductID, ProductName, UnitPrice, CategoryID):** Lưu thông tin sản phẩm.
 - **Warehouses(WarehouseID, Name, Address, CategoryID):** Lưu thông tin kho hàng.
 - **InStocks(WarehouseID, ProductID, Quantity):** Lưu số lượng tồn kho của sản phẩm tại kho.

Truy vấn:

```
SELECT *
FROM Products
WHERE (ProductID, CategoryID) IN
    (SELECT ProductID, B.CategoryID
     FROM Warehouses A, Products B
     WHERE WarehouseID = 'W01'
     AND A.CategoryID = B.CategoryID);
```

- **Mục đích:** Tìm tất cả các sản phẩm có thể được lưu trữ trong kho có mã W01.
- **Cách hoạt động:**
 - **Subquery** (SELECT ProductID, B.CategoryID FROM Warehouses A, Products B WHERE WarehouseID = 'W01' AND A.CategoryID = B.CategoryID):
 - **Kết hợp bảng Warehouses (A) và Products (B).**
 - **Lọc các dòng có WarehouseID = 'W01'** (chỉ lấy kho W01) và A.CategoryID = B.CategoryID (chỉ lấy các sản phẩm thuộc danh mục mà kho W01 hỗ trợ).
 - Trả về một quan hệ với hai cột: ProductID và CategoryID.

- **Điều kiện** (ProductID, CategoryID) IN ...:
 - Kiểm tra xem **cặp (ProductID, CategoryID)** của mỗi sản phẩm trong bảng **Products** có **nằm trong tập hợp các cặp (ProductID, CategoryID)** do subquery trả về hay không.
 - Nếu có, sản phẩm đó được chọn vì nó thuộc danh mục mà kho W01 có thể lưu trữ.
- **Kết quả:** Truy vấn trả về thông tin của các sản phẩm (từ bảng Products) có thể được lưu trữ trong kho W01.
- **Giả sử:**
 - Bảng Warehouses: (WarehouseID: 'W01', Name: 'Kho A', Address: 'Hà Nội', CategoryID: 1)
 - Bảng Products: (ProductID: 101, ProductName: 'Sữa', UnitPrice: 10, CategoryID: 1),
(ProductID: 102, ProductName: 'Bánh', UnitPrice: 5, CategoryID: 2)
 - Subquery trả về: {(101, 1)} (vì chỉ sản phẩm 101 có CategoryID = 1 khớp với kho W01).
 - Truy vấn sẽ trả về sản phẩm (101, 'Sữa', 10, 1) vì cặp (101, 1) nằm trong kết quả của subquery.
- **Lưu ý:**
 - Tuple (ProductID, CategoryID) phải có **số lượng thành phần bằng** số cột của quan hệ trong subquery (ở đây là 2 cột: ProductID, CategoryID).
 - Toán tử IN kiểm tra sự tồn tại của tuple trong quan hệ R. Tương tự, có thể dùng <> ANY để kiểm tra tuple không khớp với bất kỳ dòng nào trong R.

10. Correlated Subqueries (truy vấn con tương quan)

- Là một truy vấn con được **thực thi nhiều lần**, mỗi lần tương ứng với một dòng của truy vấn ngoài (outer query).
- Correlated subquery giống như một **vòng lặp**: Với mỗi dòng trong truy vấn ngoài, SQL sẽ chạy lại subquery, sử dụng giá trị của dòng đó để lọc dữ liệu trong subquery.
- Khác với subquery thông thường (chỉ thực thi một lần và trả về kết quả cố định), correlated subquery **sử dụng giá trị từ truy vấn ngoài** để lọc hoặc tính toán trong truy vấn con.
- Điều này làm cho correlated subquery **phụ thuộc** vào truy vấn ngoài, khác với subquery độc lập (non-correlated subquery).
- **Ví dụ trong lược đồ Warehouse Management:** Tìm thông tin về kho hàng và sản phẩm mà số lượng tồn kho của sản phẩm đó trong kho lớn hơn 30.

```
SELECT A.*, B.*
FROM Warehouses A, Products B
WHERE A.CategoryID = B.CategoryID
      AND 30 < (SELECT Quantity
                FROM InStocks
                WHERE WarehouseID = A.WarehouseID
                  AND ProductID = B.ProductID)
ORDER BY WarehouseID, ProductID;
```

- **Giải thích truy vấn:**
 - **Truy vấn ngoài:**
 - Kết hợp bảng Warehouses (A) và Products (B) với điều kiện A.CategoryID = B.CategoryID (**chỉ lấy các sản phẩm thuộc danh mục mà kho hỗ trợ**).
 - Điều kiện 30 < (subquery) kiểm tra xem số lượng tồn kho của sản phẩm trong kho có lớn hơn 30 hay không.

- **Subquery:** (SELECT Quantity FROM InStocks WHERE WarehouseID = A.WarehouseID AND ProductID = B.ProductID):
 - Lấy số lượng (Quantity) từ bảng InStocks cho cặp (WarehouseID, ProductID) tương ứng với dòng hiện tại của truy vấn ngoài (A.WarehouseID, B.ProductID).
 - Subquery này được thực thi **mỗi lần** cho từng cặp (A.WarehouseID, B.ProductID) từ truy vấn ngoài.
 - Nếu subquery trả về một giá trị Quantity lớn hơn 30, điều kiện $30 < \dots$ trả về **TRUE**, và cặp kho-sản phẩm đó được chọn.
- **ORDER BY:** Sắp xếp kết quả theo WarehouseID và ProductID.
- **Kết quả:** Truy vấn trả về thông tin của kho (A.*) và sản phẩm (B.*) thỏa mãn điều kiện số lượng tồn kho lớn hơn 30.
- **Giải sử:**
 - Bảng Warehouses: (WarehouseID: 'W01', Name: 'Kho A', Address: 'Hà Nội', CategoryID: 1)
 - Bảng Products: (ProductID: 101, ProductName: 'Sữa', UnitPrice: 10, CategoryID: 1)
 - Bảng InStocks: (WarehouseID: 'W01', ProductID: 101, Quantity: 50)
 - Khi truy vấn ngoài xét cặp (W01, 101):
 - Subquery kiểm tra: SELECT Quantity FROM InStocks WHERE WarehouseID = 'W01' AND ProductID = 101 → trả về 50.
 - Điều kiện $30 < 50$ là **TRUE**, nên thông tin của kho W01 và sản phẩm 101 được chọn.
 - Kết quả: (W01, 'Kho A', 'Hà Nội', 1, 101, 'Sữa', 10, 1).
- **Lưu ý:**
 - **Tương quan:** Subquery sử dụng A.WarehouseID và B.ProductID từ truy vấn ngoài, nên nó phải được thực thi lại cho mỗi dòng của truy vấn ngoài.
 - **Hiệu suất:** **Correlated subquery có thể chậm hơn subquery thông thường vì nó chạy nhiều lần.** Trong một số trường hợp, có thể viết lại truy vấn bằng cách sử dụng JOIN để cải thiện hiệu suất

V. Ràng buộc Toàn vẹn (Integrity Constraints)

- Ràng buộc toàn vẹn là các quy tắc được áp dụng để đảm bảo tính chính xác và hợp lệ của dữ liệu trong cơ sở dữ liệu.
- Các RBTV là những yêu cầu mà tất cả thể hiện của quan hệ phải thỏa.
- Nhằm đảm bảo:
 - CSDL luôn đúng về mặt ngữ nghĩa
 - Tính nhất quán của dữ liệu
- RBTV xuất phát từ:
 - Yêu cầu quản lý thực tế: điểm, ngày giao hàng/ngày đặt hàng, ...
 - Mô hình dữ liệu (quan hệ): khóa chính, khóa ngoại
- **Các đặc trưng của Ràng buộc toàn vẹn**
 - **Nội dung:** phát biểu bằng ngôn ngữ hình thức (phép tính quan hệ, đại số quan hệ, mã giả,...)
 - **Bối cảnh:** là những quan hệ có khả năng làm cho RBTV bị vi phạm.
 - **Tầm ảnh hưởng:** là bảng 2 chiều, **xác định các thao tác ảnh hưởng (+) và thao tác không ảnh hưởng (-)** lên các quan hệ nằm trong bối cảnh.
- **Bảng Tầm ảnh hưởng có dạng:**

	Thêm	Xóa	Sửa
Quan hệ 1	+	+	- (*)
Quan hệ ...			
Quan hệ n	-	-	+ (A)

Ký hiệu + Có thể gây ra vi phạm RBTV

Ký hiệu - Không thể gây ra vi phạm RBTV

Ký hiệu + (A) Có thể gây ra vi phạm RBTV khi thao tác trên thuộc tính A

Ký hiệu - (*) Không thể gây ra vi phạm RBTV do thao tác không thực hiện được

- Giả sử có quan hệ **R(ABC)** với ràng buộc **B > C**. Điều này có nghĩa là mọi bộ (tuple) trong quan hệ R phải thỏa mãn điều kiện **t.B > t.C**. Ta nói rằng lược đồ (scheme) R có một ràng buộc toàn vẹn.

- **Context:** Quan hệ R là nơi áp dụng ràng buộc.
- **Condition:** Điều kiện $\forall t \in R, t.B > t.C$ yêu cầu mọi giá trị của thuộc tính B phải lớn hơn giá trị của thuộc tính C trong cùng một bộ.
- **Bảng ảnh hưởng (Influence Table):**
 - Bảng ảnh hưởng xác định các thao tác (Insert, Delete, Update) có thể vi phạm ràng buộc và cần được kiểm soát

	Insert	Delete	Update
R	+	-	+(B/C)

- **Insert:** Dấu **+** vì khi thêm một bộ mới vào R, nếu $B \leq C$, ràng buộc sẽ bị vi phạm. **Cần kiểm tra.**
- **Delete:** Dấu **-** vì xóa một bộ **không ảnh hưởng** đến ràng buộc $B > C$ (ràng buộc chỉ áp dụng cho các bộ hiện có).
- **Update:** Dấu **+(B/C)** vì nếu cập nhật giá trị của B hoặc C mà khiến $B \leq C$, ràng buộc sẽ bị vi phạm. **Cần kiểm tra.**
- SQL constraints are used to specify rules for the data in a table.
- Constraints are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.
- Đây là các quy tắc để đảm bảo tính chính xác và nhất quán của dữ liệu. Chúng được định nghĩa khi tạo (CREATE) hoặc sửa (ALTER) bảng.
- Constraints can be column level or table level. Column level constraints apply to a column, and table level constraints apply to the whole table.
- Cú pháp tạo bảng với ràng buộc:**

```
CREATE TABLE tableName (
    att1 datatype columnConstraint,
    att2 datatype columnConstraint,
    ...,
    CONSTRAINT tableConstraintName <Logic expression>
);
```

- Constraints could be column level or table level.**
- Column level constraints** are applied only to one column, whereas **table level constraints** are applied to the whole table.
- Các ràng buộc ở cấp bảng bao gồm:

- **UNIQUE (columnName, [...]):** Đảm bảo sự duy nhất của một hoặc nhiều cột.
- **CHECK (expression):** Đảm bảo dữ liệu thỏa mãn một biểu thức logic (ví dụ: CHECK(B > C)).
- **PRIMARY KEY (columnName, [...]):** Xác định khóa chính cho bảng.

1. NOT NULL

- Đảm bảo một cột không thể có giá trị NULL.
- This enforces a field to always contain a value, which means that you cannot insert a new record, or update a record without adding a value to this field.
- **Example:** The following SQL ensures that the "ID", "LastName", and "FirstName" columns will NOT accept NULL values when the "Persons" table is created:

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255) NOT NULL,
  Age int
);
```

2. UNIQUE

- Đảm bảo **tất cả các giá trị trong một cột là khác nhau**.
- Both the **UNIQUE** and **PRIMARY KEY** constraints provide a guarantee (sự đảm bảo) for uniqueness for a column or set of columns.
- **A PRIMARY KEY constraint automatically has a UNIQUE constraint.**
- However, you can have **many UNIQUE constraints** per table, but **only one PRIMARY KEY constraint** per table.
- The **UNIQUE** constraint allows **NULL** values, and multiple **NULL** values are allowed in a column with a UNIQUE constraint because **NULL** is considered a distinct value in SQL. However, this behavior is different from the **PRIMARY KEY** constraint, where **NULL** values are **not** allowed, as a primary key must uniquely identify each row and cannot have missing or undefined values.
- **Example 1: Creating a Table with UNIQUE Constraints**

```
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  Name VARCHAR(100),
  Email VARCHAR(100) UNIQUE,
  Country VARCHAR(50)
);
```

- In this case, each customer must have a unique email address. If you try to insert a duplicate email, SQL will raise an error.

```
INSERT INTO Customers (CustomerID, Name, Email, Country)
VALUES (1, 'John Doe', 'john.doe@example.com', 'USA');

INSERT INTO Customers (CustomerID, Name, Email, Country)
VALUES (2, 'Jane Smith', 'jane.smith@example.com', 'Canada');
```


-- This will fail because 'john.doe@example.com' already exists

```
INSERT INTO Customers (CustomerID, Name, Email, Country)
```

```
VALUES (3, 'Alice Johnson', 'john.doe@example.com', 'UK');
```

- The third insert will fail because the Email john.doe@example.com already exists in the Customers table.

- **Example 2: Using UNIQUE with Multiple Columns**

```
CREATE TABLE Orders (
```

```
    OrderID INT PRIMARY KEY,
```

```
    CustomerID INT,
```

```
    ProductID INT,
```

```
    OrderDate DATE,
```

```
    UNIQUE (CustomerID, ProductID)
```

```
);
```

- In this example, the combination of CustomerID and ProductID must be unique, meaning **a customer cannot order the same product more than once.**

- **Example 3: Checking for Unique Values Using Subqueries**

- SQL allows you to check for **uniqueness** in **subqueries**. You can use the UNIQUE keyword in a subquery to ensure that the results do not contain duplicate values.

```
SELECT CustomerID
```

```
FROM Orders
```

```
WHERE UNIQUE (
```

```
    SELECT OrderID
```

```
    FROM OrderDetails
```

```
    WHERE Orders.CustomerID = OrderDetails.CustomerID
```

```
);
```

- In this example, we check **if there are any duplicate OrderID values for each customer in the Orders table.** *If the subquery returns unique values, the CustomerID will be selected.*

3. PRIMARY KEY (Khóa chính)

- A primary key is a column or a set of columns in a table that uniquely identifies each row.
- It ensures data integrity by preventing duplicate records and null values.
- Một sự kết hợp của NOT NULL và UNIQUE → Primary keys must contain **UNIQUE values, and cannot contain NULL values.**
- Mỗi bảng chỉ có một khóa chính, dùng để xác định duy nhất mỗi bản ghi.
- Trong bảng, khóa chính này có thể bao gồm một (*simple primary key*) hoặc nhiều cột (trường) (*composite primary key*).
 - Be cautious when using composite keys. While they are useful in some scenarios, they can make queries more complex. If possible, use a simple key or generate an artificial primary key (like an ID).
- Once a primary key is established, changing it can be difficult because of the interdependencies with other tables (foreign key constraints). Always plan ahead when designing your database schema.

- **SQL PRIMARY KEY with CREATE TABLE statement:**

```
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
    ...,
    CONSTRAINT pk_constraint_name PRIMARY KEY (column1, column2, ...)
);
```

- **SQL PRIMARY KEY with ALTER TABLE statement**

```
ALTER TABLE table_name
ADD CONSTRAINT constraint_name PRIMARY KEY (column1, column2, ... column_n);
```

- **Examples**

- **Create PRIMARY KEY in SQL Example**

```
CREATE TABLE Persons (
    PersonID int NOT NULL PRIMARY KEY,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int
);
```

- **Verify SQL Primary key creation:** To verify if the primary key has been successfully created, we will try adding duplicate values in primary key column, and SQL should return an error.

```
INSERT INTO Persons VALUES
(1,"Thakur", "Aditya", 22),
(1, "Kumar", "Shubham", 21);
```

Output: Error: UNIQUE constraint failed: Persons.PersonID

- **Add PRIMARY KEY to an already existing table Example:**

```
CREATE TABLE Persons (
    PersonID int,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int
);
```

This query will add primary key to 'Persons' table

```
ALTER TABLE Persons ADD CONSTRAINT PK_Person PRIMARY KEY (PersonID);
```

- **Creating a table with a composite key:**

```
CREATE TABLE student (
    rollNumber INT,
    name VARCHAR(30),
    class VARCHAR(30),
    section VARCHAR(1),
    mobile VARCHAR(10),
    PRIMARY KEY (rollNumber, mobile)
);
```

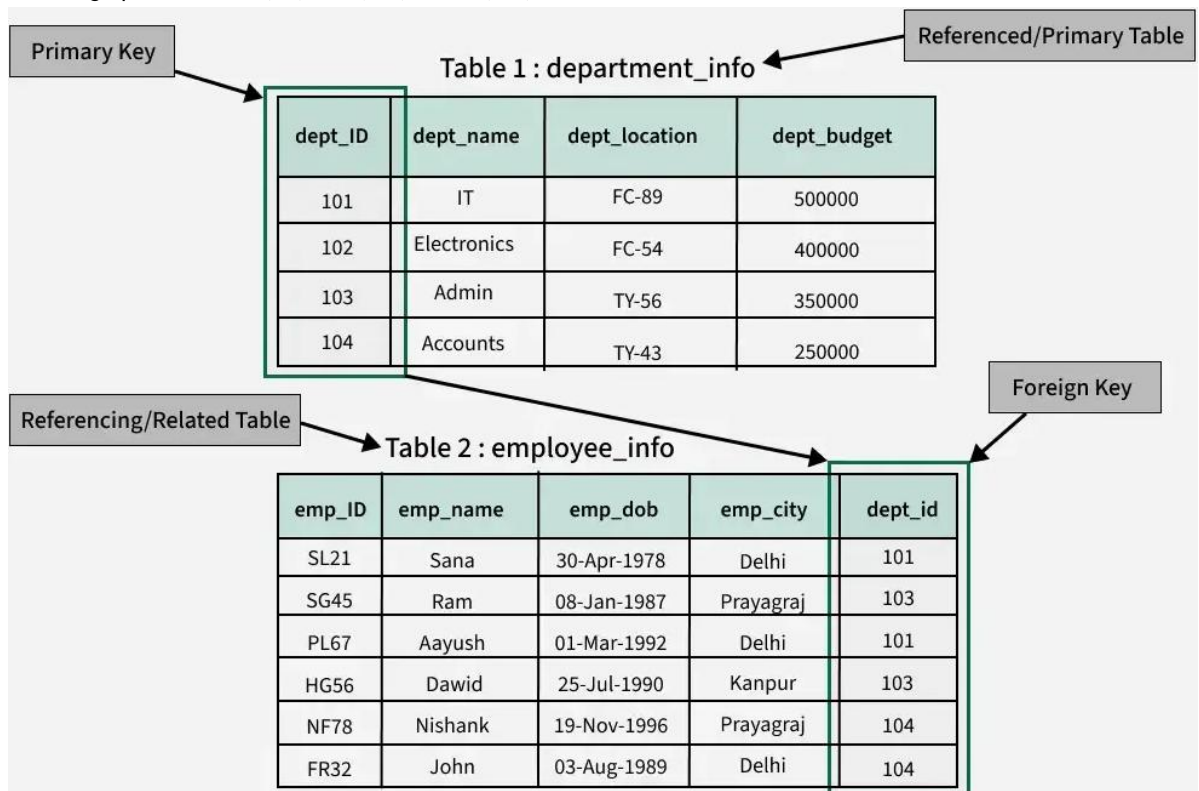
- To allow naming of a PRIMARY KEY constraint, and for defining a PRIMARY KEY constraint on multiple columns, use the following SQL syntax (**MySQL / SQL Server / Oracle / MS Access**):

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CONSTRAINT PK_Person PRIMARY KEY (ID,LastName)
);
```

- Note:** In the example above **there is only ONE PRIMARY KEY (PK_Person)**. However, **the VALUE of the primary key is made up of TWO COLUMNS (ID + LastName)**.

4. FOREIGN KEY (Khóa ngoại)

- The **FOREIGN KEY** constraint is used to **prevent actions that would destroy links between tables**.
- A **FOREIGN KEY** is a field (or collection of fields) in one table, that refers to the **PRIMARY KEY** in another table.
- Khóa ngoại không phải là khóa chính của quan hệ đó, nhưng là khóa chính của một quan hệ khác. Nó đảm bảo rằng giá trị trong khóa ngoại phải tồn tại trong khóa chính của quan hệ được tham chiếu.
- The table with the foreign key is called the **foreign table/ child table**, and the table with the primary key is called the **referenced/primary/parent table**.
- Dùng để liên kết hai bảng với nhau.
- Giá trị trong cột khóa ngoại của bảng này phải là một giá trị có trong cột khóa chính của bảng kia. Điều này đảm bảo *toàn vẹn tham chiếu*.
- Cho $R_1(XY)$, $R_2(ZX)$. Tập giá trị của khóa ngoại trong quan hệ con (R_2) phải là tập con của tập giá trị khóa chính trong quan hệ cha (R_1): $\pi X(R_2) \subseteq \pi X(R_1)$.



- **SQL FOREIGN KEY on CREATE TABLE statement**

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    ...,
    CONSTRAINT fk_constraint_name FOREIGN KEY (column1, column2, ...)
    REFERENCES parent_table(column1, column2, ...)
);
```

- **SQL FOREIGN KEY on ALTER TABLE statement**

```
ALTER TABLE table_name
ADD CONSTRAINT fk_constraint_name FOREIGN KEY (column1, column2, ...)
REFERENCES parent_table(column1, column2, ...);
```

- **Viết đầy đủ theo hai quan hệ R_1, R_2 :**

```
CREATE TABLE R2 (
    Z datatype,
    X datatype,
    FOREIGN KEY (X) REFERENCES R1(X)
    [ON UPDATE <CASCADE / SET NULL>]
    [ON DELETE <CASCADE / SET NULL>]
);
```

- **ON UPDATE / ON DELETE:** Xác định hành vi khi cập nhật hoặc xóa dữ liệu trong bảng cha (R_1):
 - **CASCADE:** Thay đổi hoặc xóa dữ liệu trong R_1 sẽ tự động cập nhật/xóa các bản ghi tương ứng trong R_2 .
 - **SET NULL:** Thay đổi hoặc xóa dữ liệu trong R_1 sẽ đặt giá trị khóa ngoại trong R_2 thành NULL.
 - **Default (REJECT):** Từ chối thao tác nếu vi phạm ràng buộc khóa ngoại.

- **Ví dụ 1:**

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50) NOT NULL
);

CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    OrderNumber INT NOT NULL,
    CustomerID INT,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

INSERT INTO Customers (CustomerID, CustomerName)
VALUES (1, 'John'), (2, 'Jane'), (3, 'Bob');

INSERT INTO Orders (OrderID, OrderNumber, CustomerID)
VALUES (1, 101, 1), (2, 102, 2), (3, 103, 3);
```

The following query will create the 'customers' and 'orders' table.

Customers		Orders		
CustomerID	CustomerName	OrderID	OrderNumber	CustomerID (Foreign Key)
1	John			
2	Jane	1	101	1
3	Bob	2	102	2
		3	103	3

- **Example 1.1: Insert Value in Foreign Key Table:** If a corresponding value in the foreign table doesn't exist, a record in the child table cannot be inserted.

```
INSERT INTO Orders (OrderID, OrderNumber, CustomerID)
VALUES (4, 104, 4);
```

Output: Error: FOREIGN KEY constraint failed.

- **Example 1.2: Delete a value in Foreign Key Table:** When a record in the master table is deleted and the corresponding record in the child table exists, an error message is displayed and prevents the **DELETE** operation from going through.

```
DELETE FROM Customers WHERE CustomerID = "3";
```

Output: Error: FOREIGN KEY constraint failed

- **Ví dụ 2:**

```
CREATE TABLE ShipperDetail (
    ShipperID Text,
    ProductID Text,
    Quantity INT,
    PRIMARY KEY (ShipperID, ProductID),
    FOREIGN KEY (ShipperID) REFERENCES Shippers(ShipperID)
    ON DELETE SET NULL ON UPDATE CASCADE,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
    ON DELETE SET NULL ON UPDATE CASCADE
);
```

- ShipperID và ProductID là khóa ngoại, tham chiếu đến khóa chính của bảng **Shippers** và **Products**.
- Nếu ShipperID trong **Shippers** bị xóa, ShipperID trong **ShipperDetail** sẽ được đặt thành NULL.
- Nếu ShipperID trong **Shippers** được cập nhật, ShipperID trong **ShipperDetail** sẽ tự động cập nhật theo (CASCADE).

5. CHECK

- Đảm bảo các giá trị trong một cột thỏa mãn một điều kiện cụ thể.
- The **CHECK** constraint is used to **limit the value range** that can be placed in a column.
- If you define a **CHECK** constraint **on a column** it will **allow only certain values** for this column.
- If you define a **CHECK** constraint **on a table** it can **limit the values in certain columns based on values in other columns** in the row.

- **Syntax:**

- **CHECK constraint at the column level:**

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype CHECK (condition),
    ...
);
```

- **CHECK constraint at the table level:**

```
CREATE TABLE tableName (
    ...,
    CHECK(expression)
);
```

- **Example 1: Applying CHECK on a Single Column:** we create a **Customers table** with the **Age column** that must contain values between 18 and 120. The CHECK constraint ensures that no invalid age is inserted into the table.

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT CHECK (Age >= 18 AND Age <= 120)
);
```

-- Valid insert

```
INSERT INTO Customers (CustomerID, Name, Age)
VALUES (1, 'John Doe', 25);
```

-- Invalid insert

```
INSERT INTO Customers (CustomerID, Name, Age)
VALUES (2, 'Jane Smith', 15); -- This will fail due to the CHECK constraint
```

- The Age column has a CHECK constraint that ensures the value must be between 18 and 120. If you attempt to insert an age outside this range, the database will throw an error.

- **Example 2: CHECK Constraint with Multiple Columns:** We can also use the CHECK constraint across multiple columns. For instance, let's say we have an **Employee table**, and we want to ensure that **the Salary is positive and the Age is greater than or equal to 18**.

```
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT,
    Salary DECIMAL(10, 2),
    CHECK (Age >= 18 AND Salary > 0)
);
```

-- Valid insert

```
INSERT INTO Employee (EmployeeID, Name, Age, Salary)
VALUES (1, 'Alice Johnson', 30, 50000);
```

-- Invalid insert (age < 18)

```
INSERT INTO Employee (EmployeeID, Name, Age, Salary)
```

```
VALUES (2, 'Bob Lee', 16, 45000);
```

 – This will fail due to the CHECK constraint

- The CHECK constraint ensures that both conditions are satisfied. The employee must be at least 18 years old, and the salary must be greater than 0. This kind of constraint is useful when multiple columns are involved in the rule.

- **Example 3: Adding a CHECK Constraint with ALTER TABLE:** We can add a CHECK constraint to an existing table using the ALTER TABLE statement.

```
ALTER TABLE Employee
```

```
ADD CONSTRAINT chk_salary CHECK (Salary >= 30000);
```

- This adds a **CHECK constraint named chk_salary** to the Employee table, ensuring that the Salary column has a **minimum value of 30,000**. If you attempt to insert or update a record with a salary lower than 30,000, the operation will fail.

- **Lưu ý**

- CHECK **không được dùng để kiểm tra dữ liệu giữa các dòng hoặc bảng khác**. Nó chỉ áp dụng trên từng dòng riêng lẻ.
- Nếu muốn kiểm tra điều kiện liên quan đến **dữ liệu của bảng khác**, bạn phải dùng **trigger** hoặc logic trong ứng dụng.

6. DEFAULT

- Cung cấp một giá trị mặc định cho cột khi không có giá trị nào được chỉ định.
- If a column has a **DEFAULT constraint** and no value is explicitly provided during the insertion of a record, the database will automatically insert the default value defined for that column.
- **Syntax:**

```
CREATE TABLE table_name (  
    column1 datatype DEFAULT default_value,  
    column2 datatype DEFAULT default_value,  
    ...  
);
```

- The DEFAULT constraint is optional. Not all columns require a default value.
- You cannot use the **DEFAULT** constraint to assign **NULL** as a default value. For null values, you can either omit the value or explicitly specify NULL.
- **Ví dụ 1:**

```
CREATE TABLE Geeks (  
    ID INT NOT NULL,  
    Name VARCHAR(255),  
    Age INT,  
    Location VARCHAR(255) DEFAULT 'Noida'  
);
```

-- Explicit value

```
INSERT INTO Geeks (ID, Name, Age, Location) VALUES (4, 'Mira', 23, 'Delhi');
```

-- Using the DEFAULT constraint

```
INSERT INTO Geeks (ID, Name, Age, Location) VALUES (5, 'Hema', 27);
```

-- Explicit value again

```
INSERT INTO Geeks (ID, Name, Age, Location) VALUES (6, 'Neha', 25, 'Delhi');
```

-- Using DEFAULT constraint again

```
INSERT INTO Geeks (ID, Name, Age, Location) VALUES (7, 'Khushi', 26, DEFAULT);
```

Output:

ID	Name	Age	Location
4	Mira	23	Delhi
5	Hema	27	Noida
6	Neha	25	Delhi
7	Khushi	26	Noida

- **Dropping the DEFAULT Constraint:** If you no longer want a column to use a default value, you can drop the DEFAULT constraint. This will only apply to new rows and will not affect existing data in the table.

- **Syntax:**

```
ALTER TABLE tablename  
ALTER COLUMN columnname  
DROP DEFAULT;
```

- **Ví dụ 2:**

```
ALTER TABLE Geeks  
ALTER COLUMN Location  
DROP DEFAULT;
```

Let us add 2 new rows in the Geeks table :

```
INSERT INTO Geeks VALUES (8, 'Komal', 24, 'Delhi');  
INSERT INTO Geeks VALUES (9, 'Payal', 26, NULL);
```

- **Note** - Dropping the default constraint will not affect the current data in the table, it will only apply to new rows.

```
Select * from Geeks;
```

Output:

ID	Name	Age	Location
4	Mira	23	Delhi
5	Hema	27	Noida
6	Neha	25	Delhi
7	Khushi	26	Noida
8	Komal	24	Delhi
9	Payal	26	NULL

VI. Views

- **CREATE TABLE** statement create table in database, it **stores in physical**.
- View là một **bảng ảo** dựa trên tập kết quả của một câu lệnh SQL, **(virtual) views** → do not exist physically.
- Nó chứa các hàng và cột, giống như một bảng thực. Các trường trong view là các trường từ một hoặc nhiều bảng thực trong cơ sở dữ liệu.
- View giống như một "lối tắt" hay một "báo cáo đã được lưu". Nó là một câu lệnh SELECT được đặt tên và lưu lại, bạn có thể truy vấn nó như một bảng thật.
- **Lợi ích:**
 - **Đơn giản hóa:** Che giấu sự phức tạp của các câu lệnh JOIN dài.
 - **Bảo mật:** Giới hạn quyền truy cập của người dùng vào một tập hợp dữ liệu cụ thể.

Cú pháp:

Tạo View

```
CREATE VIEW ten_view AS
SELECT cot1, cot2, ... FROM ten_bang [WHERE dieu_kien];
```

Xóa View

```
DROP VIEW ten_view;
```

- **Note:** A view always shows up-to-date data! The database engine recreates the view, every time a user queries it.
- **Ví dụ:** Tạo 2 bảng như sau:

StudentDetails			StudentMarks			
S_ID	NAME	ADDRESS	ID	NAME	MARKS	AGE
1	Harsh	Kolkata	1	Harsh	90	19
2	Ashish	Durgapur	2	Suresh	50	20
3	Pratik	Delhi	3	Pratik	80	19
4	Dhanraj	Bihar	4	Dhanraj	95	21
5	Ram	Rajasthan	5	Ram	85	18

Example 1: Creating a Simple View from a Single Table:

<pre>CREATE VIEW DetailsView AS SELECT NAME, ADDRESS FROM StudentDetails WHERE S_ID < 5;</pre> Use the below query to retrieve the data from this view <pre>SELECT * FROM DetailsView;</pre>	<table> <tr> <th>NAME</th><th>ADDRESS</th></tr> <tr> <td>Harsh</td><td>Kolkata</td></tr> <tr> <td>Ashish</td><td>Durgapur</td></tr> <tr> <td>Pratik</td><td>Delhi</td></tr> <tr> <td>Dhanraj</td><td>Bihar</td></tr> </table>	NAME	ADDRESS	Harsh	Kolkata	Ashish	Durgapur	Pratik	Delhi	Dhanraj	Bihar		
NAME	ADDRESS												
Harsh	Kolkata												
Ashish	Durgapur												
Pratik	Delhi												
Dhanraj	Bihar												
<pre>CREATE VIEW StudentNames AS SELECT S_ID, NAME FROM StudentDetails ORDER BY NAME;</pre> If we now query the view as, <pre>SELECT * FROM StudentNames;</pre>	<table> <tr> <th>S_ID</th><th>NAMES</th></tr> <tr> <td>2</td><td>Ashish</td></tr> <tr> <td>4</td><td>Dhanraj</td></tr> <tr> <td>1</td><td>Harsh</td></tr> <tr> <td>3</td><td>Pratik</td></tr> <tr> <td>5</td><td>Ram</td></tr> </table>	S_ID	NAMES	2	Ashish	4	Dhanraj	1	Harsh	3	Pratik	5	Ram
S_ID	NAMES												
2	Ashish												
4	Dhanraj												
1	Harsh												
3	Pratik												
5	Ram												

- **Example 2: Creating a View From Multiple Tables**

```
CREATE VIEW MarksView AS
SELECT StudentDetails.NAME,
StudentDetails.ADDRESS, StudentMarks.MARKS
FROM StudentDetails, StudentMarks
WHERE StudentDetails.NAME = StudentMarks.NAME;
```

To display data of View MarksView:
 SELECT * FROM MarksView;

NAME	ADDRESS	MARKS
Harsh	Kolkata	90
Pratik	Delhi	80
Dhanraj	Bihar	95
Ram	Rajasthan	85

- **A view can be updated** with the CREATE OR REPLACE VIEW statement, without affecting the data.

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

- **Ví dụ:** We can use the **CREATE OR REPLACE VIEW** statement to add or replace fields from a view if we want to update the view **MarksView** and **add the field AGE** to this View from **StudentMarks** Table, we can do this.

```
CREATE OR REPLACE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS, StudentMarks.AGE
FROM StudentDetails, StudentMarks
WHERE StudentDetails.NAME = StudentMarks.NAME;
```

If we fetch all the data from MarksView now as:

```
SELECT * FROM MarksView;
```

NAME	ADDRESS	MARKS	AGE
Harsh	Kolkata	90	19
Pratik	Delhi	80	19
Dhanraj	Bihar	95	21
Ram	Rajasthan	85	18

- **Inserting Data into Views:** We can insert a row in a View in the same way as we do in a table. We can use the INSERT INTO statement of SQL to insert a row in a View.
 - In the below example, we will insert a new row in the View DetailsView which we have created above in the example of "**creating views from a single table**".

```
INSERT INTO DetailsView(NAME, ADDRESS)
VALUES("Suresh", "Gurgaon");
```

If we fetch all the data from DetailsView now as,
 SELECT * FROM DetailsView;

NAME	ADDRESS
Harsh	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar
Suresh	Gurgaon

- **Deleting a row from a View:** Deleting rows from a view is also as simple as deleting rows from a table. We can use the DELETE statement of SQL to delete rows from a view. Also deleting a row from a view first deletes the row from the actual table and the change is then reflected in the view.
 - In this example, we will delete the last row from the view DetailsView which we just added in the above example of inserting rows.

<pre>DELETE FROM DetailsView WHERE NAME="Suresh";</pre> If we fetch all the data from DetailsView now as, <pre>SELECT * FROM DetailsView;</pre>	<table> <thead> <tr> <th>NAME</th><th>ADDRESS</th></tr> </thead> <tbody> <tr> <td>Harsh</td><td>Kolkata</td></tr> <tr> <td>Ashish</td><td>Durgapur</td></tr> <tr> <td>Pratik</td><td>Delhi</td></tr> <tr> <td>Dhanraj</td><td>Bihar</td></tr> </tbody> </table>	NAME	ADDRESS	Harsh	Kolkata	Ashish	Durgapur	Pratik	Delhi	Dhanraj	Bihar
NAME	ADDRESS										
Harsh	Kolkata										
Ashish	Durgapur										
Pratik	Delhi										
Dhanraj	Bihar										

- Trong một số **trường hợp đơn giản**, bạn có thể **INSERT, UPDATE hoặc DELETE** dữ liệu thông qua VIEW, tuy nhiên điều này **phụ thuộc vào cấu trúc của view**.
- **Những trường hợp khó hoặc không thể cập nhật:**
 - View có dùng GROUP BY, DISTINCT, AGGREGATE FUNCTION (SUM, AVG, ...)
 - View có JOIN nhiều bảng
 - View có UNION, LIMIT, OFFSET...

VII. Renaming

- The AS command is used to **rename a column or table with an alias**.
- An alias only exists **for the duration of the query**.
- **Alias for Columns**

```
SELECT column_name AS alias_name
FROM table_name;
```

- **column_name:** The original name of the column in the table.
- **alias_name:** The temporary name you want to assign to the column in the output.

- **Alias for Tables**

```
SELECT column_name
FROM table_name AS alias_name;
```

- **table_name:** The original name of the table.
- **alias_name:** The temporary name assigned to the table.

- **Ví dụ:**

Orders				Customers	
OrderID	CustomerID	OrderDate	TotalAmount	CustomerID	CustomerName
1	101	2024-08-01	250	101	Alice
2	102	2024-08-02	300	102	Bob
3	103	2024-08-03	150	103	Charlie
4	101	2024-08-04	450	104	Dana
5	104	2024-08-05	200		

- **Example 1: Alias for Columns:** To select data from the Orders table and rename the columns for clarity, we can use aliases.

SELECT OrderID AS "Order Number", CustomerID AS "Client ID", OrderDate AS "Date", TotalAmount AS "Amount" FROM Orders;	Order Number	Client ID	Date	Amount
	1	101	2024-08-01	250
	2	102	2024-08-02	300
	3	103	2024-08-03	150
	4	101	2024-08-04	450
	5	104	2024-08-05	200

- **Example 2: Using Aliases With a Space Character:** If we want to include **spaces** in our **column aliases**, we must **enclose** the **alias name** in **quotes**.

SELECT CustomerID AS "Customer ID", TotalAmount AS "Order Amount" FROM Orders;	Customer ID	Order Amount
	101	250
	102	300
	103	150
	101	450
	104	200

- **Example 3: Concatenate Columns:** To create a full name by concatenating **FirstName** and **LastName** columns from the **Customers** table (assuming it has **FirstName** and **LastName** columns), we can use concatenation with an alias.

SELECT CONCAT(FirstName, ' ', LastName) AS "Full Name" FROM Customers;	Full Name
	Alice Smith
	Bob Brown
	Charlie Davis
	Dana White

- **Example 4:** We can give a view's attributes in new names.

CREATE VIEW ProductInStock(Product, TotalQuantity) AS SELECT ProductID, SUM(Quantity) FROM Instocks GROUP BY ProductID;
--

- ProductInStock: là **tên view** bạn đang tạo.
- (Product, TotalQuantity): là **tên cột mới** trong view (bạn có thể đặt lại tên tùy ý).
- SELECT ProductID, SUM(Quantity): truy vấn lấy tổng số lượng theo từng sản phẩm.
- GROUP BY ProductID: nhóm dữ liệu theo ProductID.

VIII. Indexes (Chỉ mục)

- Hãy tưởng tượng Index giống như mục lục của một cuốn sách. Thay vì phải lật từng trang để tìm một chương, bạn chỉ cần xem mục lục. Tương tự, **Index giúp CSDL tìm kiếm dữ liệu trên một cột nhanh hơn** rất nhiều.
- Index là một cấu trúc dữ liệu đặc biệt giúp tăng tốc độ truy vấn dữ liệu đáng kể.
- **Người dùng không thể thấy các chỉ mục**, chúng chỉ được sử dụng để tăng tốc độ tìm kiếm/truy vấn.
- Mỗi hàng (row) trong bảng có một rowID (số thứ tự duy nhất) để nhận dạng. **Chỉ mục có thể được xem như một danh sách các cặp (giá trị cột, rowID)** được sắp xếp theo thứ tự của cột được chọn.
- **Ưu điểm:**
 - Tăng tốc độ cho các truy vấn sử dụng **SELECT, WHERE, JOIN**, hoặc **GROUP BY** trên các cột được lập chỉ mục.

- Giảm thời gian tìm kiếm dữ liệu bằng cách sử dụng cấu trúc dữ liệu tối ưu (thường là cây B+ hoặc cấu trúc tương tự).
- **Nhược điểm:**
 - Làm chậm các thao tác **INSERT**, **UPDATE**, và **DELETE** vì chỉ mục cần được cập nhật mỗi khi dữ liệu trong bảng thay đổi.
 - Tốn thêm không gian lưu trữ cho cấu trúc chỉ mục.
- **Tính chất:**
 - Chỉ mục có thể được tạo hoặc xóa mà không ảnh hưởng đến dữ liệu trong bảng.
 - Chỉ mục không phải là dữ liệu gốc, mà chỉ là một cấu trúc hỗ trợ.
- **Cú pháp:**
 - **Tạo Index trên một cột/ nhiều cột**

```
CREATE INDEX ten_index ON ten_bang (ten_cot);
CREATE INDEX ten_index ON ten_bang (column1, column2, ...);
```

- **Tạo Index duy nhất, đảm bảo các giá trị trong cột (hoặc tổ hợp cột) không trùng lặp.**

```
CREATE UNIQUE INDEX ten_index ON ten_bang (ten_cot);
```

- **Xóa Index khỏi cơ sở dữ liệu. Thao tác này không ảnh hưởng đến dữ liệu trong bảng.**

```
DROP INDEX ten_index;
```

- **Ví dụ:** Bảng **Categories**:

rowID	CategoryID	CategoryName	Description
1	c1	Rice	...
2	c2	Electronic	...
3	c3	Flower	...

```
CREATE INDEX CategoryNameIndex ON Categories(CategoryName ASC);
```

- Tạo chỉ mục trên cột CategoryName của bảng Categories, sắp xếp theo thứ tự tăng dần (ASC).
- **Kết quả:** Chỉ mục lưu trữ các giá trị của CategoryName (Electronic, Flower, Rice) cùng với rowID tương ứng (2, 3, 1).

CategoryName	rowID
Electronic	2
Flower	3
Rice	1

- **Chỉ mục nên được tạo trong các trường hợp sau:**
 - **Cột có nhiều giá trị khác nhau** (high cardinality): Ví dụ, cột CategoryName chứa các giá trị như Rice, Electronic, Flower (ít trùng lặp).
 - **Cột có ít giá trị NULL:** Chỉ mục hiệu quả hơn khi hầu hết các hàng đều có giá trị.
 - **Cột thường được sử dụng trong điều kiện truy vấn:**
 - Trong mệnh đề **WHERE** (ví dụ: WHERE CategoryName = 'Rice').
 - Trong điều kiện **JOIN** (ví dụ: nối bảng dựa trên CategoryID).
 - Trong các thao tác **GROUP BY** hoặc **ORDER BY**.

- **Không nên tạo chỉ mục trong các trường hợp sau:**

- **Bảng nhỏ:** Nếu bảng chỉ có vài bản ghi, việc lập chỉ mục không mang lại lợi ích đáng kể vì truy vấn đã đủ nhanh.
- **Cột hiếm khi được sử dụng trong truy vấn:** Nếu cột không xuất hiện trong **WHERE**, **JOIN**, hoặc **GROUP BY**, chỉ mục sẽ không được dùng và chỉ tốn tài nguyên.
- **Cột thường xuyên bị cập nhật:** Các thao tác **INSERT**, **UPDATE**, hoặc **DELETE** trên cột có chỉ mục sẽ chậm hơn do phải cập nhật cả chỉ mục.

IX. Auto Increment

- Auto-increment **allows a unique number to be generated automatically** when a new record is inserted into a table.
- **Often this is the primary key field** that we would like to be created automatically every time a new record is inserted.

- **MySQL**

- The following SQL statement defines the "Personid" column to be an auto-increment primary key field in the "Persons" table:

```
CREATE TABLE Persons (  
    Personid int NOT NULL AUTO_INCREMENT,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    PRIMARY KEY (Personid)  
);
```

- **MySQL uses the AUTO_INCREMENT** keyword to perform an auto-increment feature.
- By default, **the starting value for AUTO_INCREMENT is 1, and it will increment by 1** for each new record.
- To let the AUTO_INCREMENT sequence **start with another value**, use the following SQL statement:

```
ALTER TABLE Persons AUTO_INCREMENT=100;
```

- To insert a new record into students table, **we will not specify the value of Personid as it will be added automatically.**

- **PostgreSQL**

- In PostgreSQL, the **SERIAL** keyword is used for auto-incrementing a column. This is a shorthand for creating an integer column that automatically increments.
- **Example:** We will create Students table with fields **Student_ID**, **First_Name**, **Last_Name**, we will auto generate Student_ID by using **auto increment** and will make it Primary Key for the table.

```
CREATE TABLE Students(  
    Student_ID int SERIAL PRIMARY KEY,  
    First_Name varchar(255),  
    Last_Name varchar(255)  
);
```

- **Inserting Data with Auto Increment:** To insert a new record into students table, **we will not specify the value of Student_ID as it will be added automatically.**

```
INSERT INTO Students(First_Name, Last_Name )  
VALUES ('Anish', 'Jain');
```

```
INSERT INTO Students(First_Name, Last_Name )  
VALUES ('Akshita', 'Sharma');
```

```
INSERT INTO Students(First_Name, Last_Name )  
VALUES ('Shruti', 'Sogani');
```

Student_ID	First_Name	Last_Name
1	Anish	Jain
2	Akshita	Sharma
3	Shruti	Sogani